

CIPO: Counterfactual Imagination Policy Optimization for Adaptive Tool Granularity Selection

Yu Li¹, Yunlu Wan¹, Zijian Zhu¹, Han Luo¹, Chao Ren², Long-Fei Li³, and Lei Feng¹

¹School of Computer Science and Engineering, Southeast University, Nanjing, China

²KTH Royal Institute of Technology ³Huawei Noah’s Ark Lab, China

Abstract

Large language model (LLM) agents solve complex tasks through multi-step interactions with external tools. These interactions often contain recurring local tool sequences. Treating such sequences as composite “Skills” can shorten tool-use trajectories and reduce repeated low-level decisions. However, when atomic tools and composite skills coexist, skill use becomes a policy problem: the agent must decide whether the current state requires atomic fine control or skill-level abstraction. In this paper, we argue that effective skill use should be studied as adaptive tool granularity selection. The most direct training signal for this problem is to compare the consequences of atomic and skill choices available from the same state. Based on this view, we propose CIPO, a Counterfactual Imagination Policy Optimization framework for adaptive tool granularity. CIPO constructs executable skills through budget-constrained mining of successful tool-use trajectories and trains granularity decisions with counterfactual branch rollouts. For each base rollout, CIPO branches at the first eligible granularity decision and replaces the chosen action with a feasible atomic or skill alternative. The paired outcome difference serves as a supplementary reward for policy optimization. Experiments across multiple benchmarks and model backbones show that CIPO improves task success and decision efficiency over baselines. Further analyses show that CIPO learns effective skill use by improving the choice between atomic tools and composite skills based on the current state, without simply increasing skill frequency.

1. Introduction

Large language model (LLM) agents use external tools to solve complex tasks involving information retrieval (Nakano et al., 2021, Min et al., 2026b,a), code execution (Gao et al., 2023), file manipulation (Chen et al., 2023), and API interaction (Patil et al., 2024, Yao et al., 2023, Schick et al., 2023, Li et al., 2023, Qin et al., 2024, Zeng et al., 2024, Li et al., 2026b, 2024a). As these tasks become more complex, tool use often unfolds as long trajectories in which the agent repeatedly selects tools under changing intermediate states (Liu et al., 2023, Zhou et al., 2024, Yao et al., 2024, Lu et al., 2025, Li et al., 2025). Such trajectories commonly contain reusable local routines, including metadata retrieval followed by content reading, question parsing followed by answer extraction, and directory listing followed by file reading. These recurring sequences indicate that long-horizon tool use is not merely a chain of isolated tool invocations. It also contains reusable procedures that can be packaged as composite “Skills” (Sutton et al., 1999, Liang et al., 2023, Wang et al., 2023, Jiang et al., 2026), allowing several low-level tool invocations to be compressed into a single higher-level action.

Although recent work has explored different forms of reuse processes in LLM agents, existing approaches do not fully address how these reusable procedures should be constructed, executed, and selected during policy learning. Memory-based methods retrieve past trajectories, reflections, or task records to guide

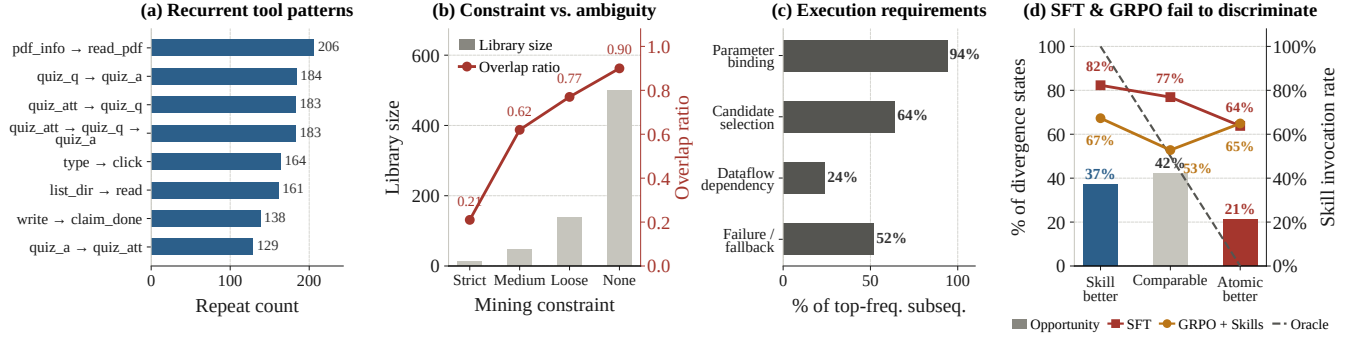


Figure 1: Pilot analyses on tool-use granularity. (a) Successful TOOLATHLON trajectories contain recurrent tool patterns. (b) Looser mining constraints increase both the skill library size and candidate overlap, motivating budget-constrained skill construction. (c) Frequent subsequences often require runtime execution support, including parameter binding, candidate selection, dataflow handling, and failure recovery. (d) Granularity preference varies across states, while SFT and “GRPO + Skills” fail to follow the oracle trend, suggesting the need for explicit granularity training.

the current agent (Park et al., 2023, Shinn et al., 2023, Zhao et al., 2024, Packer et al., 2023), but the retrieved experience usually remains in the context and does not change the action space (Xu et al., 2025a, Chhikara et al., 2025, Fang et al., 2025). Workflow reuse methods store or synthesize reusable procedures from previous tasks (Wang et al., 2025, Chen et al., 2024, Li et al., 2024b, Zhang et al., 2025), but these procedures are often treated as fixed guidance rather than actions that compete with atomic tools at each decision state. Tool merging and compilation methods combine tools or tool invocations into higher-level interfaces (Abuzakuk et al., 2026, Kim et al., 2024, Singh et al., 2024), but direct merging can create a large number of overlapping candidates. Recent skill-learning methods further suggest that tool-use procedures can be reused as skills (Xia et al., 2026, Lu et al., 2026, Ni et al., 2026, Jiang et al., 2026), yet they often leave open the question of how an agent should decide whether the current state calls for an atomic tool or a composite skill. These limitations suggest that reusable tool procedures introduce not only a construction problem, but also an execution problem and a policy problem.

To examine these challenges, we conduct a series of pilot analyses. As shown in Figure 1(a), recurrent tool patterns are common in successful TOOLATHLON (Li et al., 2025) trajectories, with several subsequences appearing over 100 times. This result supports the basic premise that skill construction can exploit repeated local structure. However, Figure 1(b) shows that relaxing mining constraints rapidly increases both the size of the skill library and the overlap among mined candidates. This pattern indicates that unconstrained tool merging can produce an ambiguous action space, where many candidates express similar procedures. Figure 1(c) further shows that frequent subsequences are rarely executable as fixed pipelines. Most of them require runtime decisions, including parameter binding, candidate selection, dataflow handling, or failure recovery. Finally, Figure 1(d) shows that granularity preference is state-dependent. Skills are better in 37% of the examined states, atomic tools are better in 21%, and the remaining states are comparable. However, SFT and “GRPO + Skills” do not clearly separate these categories. Their skill invocation rates remain far from the oracle trend, which suggests that standard training can expose the model to skills but does not provide sufficient feedback for state-dependent granularity selection.

These findings imply three requirements for skill-augmented tool use. First, skill construction should be constrained, because frequent subsequences must be selected without causing uncontrolled library growth or severe overlap. Second, mined subsequences must be instantiated as executable skills, because useful skills need parameter interfaces, selection control, execution traces, and interruption handling rather than fixed symbolic pipelines. Third, policy training must teach the agent when to use each granularity. A composite skill can improve efficiency when its internal procedure matches the current subtask, but it can also hide

useful intermediate observations or execute unnecessary steps when finer control is needed. We formulate this third requirement as adaptive tool granularity selection (Ning et al., 2024). Given a decision state where an atomic action and a composite skill are feasible, the policy should choose the granularity that better supports the remaining trajectory. The most direct training signal for this choice is a comparison between the consequences of atomic and skill actions available from the same state (Xi et al., 2026, Liu et al., 2025).

Based on these requirements, we propose CIPO, a Counterfactual Imagination Policy Optimization framework for adaptive tool granularity. CIPO first constructs a compact set of executable skills from successful tool-use trajectories through budget-constrained BPE-style (Sennrich et al., 2016) mining and trace-aware instantiation. The mining stage selects recurring tool subsequences under explicit constraints, while the instantiation stage converts each selected subsequence into a callable skill with parameter binding, selection control, execution tracing, and soft interruption. This design preserves atomic tools for fine control while adding composite skills for reusable procedures.

CIPO then extends GRPO (Shao et al., 2024) to train the policy to choose between atomic tools and composite skills. For each base rollout, CIPO creates a counterfactual branch at the first eligible granularity decision in the base rollout and replaces the chosen action with a feasible atomic or skill alternative. The branch continues under the same policy, which keeps the comparison tied to the same decision context. The paired outcome difference is used as a supplementary reward in GRPO, while the original task reward remains the main training signal.

We evaluate CIPO across multiple tool-use benchmarks with Qwen2.5-7B and Llama3.1-8B backbones. The results show that CIPO improves task success and decision efficiency over baselines. Further analyses show that the improvement does not come from simply increasing skill frequency. Instead, CIPO learns more effective skill use by improving the policy’s ability to select the appropriate tool granularity under the current state.

Our contributions are summarized as follows:

- We formulate adaptive tool granularity selection as a key policy problem for skill-augmented LLM agents, showing through pilot analyses that SFT and GRPO with access to skills do not reliably decide when to use atomic tools or composite skills based on their relative benefits in the current state.
- We introduce a budget-constrained BPE-style skill construction pipeline that turns recurring tool-use patterns into executable skills with parameter binding, execution traces, and soft interruption.
- We propose CIPO, a counterfactual policy optimization framework that improves task success, decision efficiency, and effective skill use across multiple benchmarks and backbones.

2. Related Work

Tool-Augmented LLM Agents. Recent LLM agents use external tools to access information, execute code, and call real-world APIs (Nakano et al., 2021, Li et al., 2023, Zeng et al., 2024, Yang et al., 2026a,b, Li et al., 2026c, Yang et al., 2026c). ReAct interleaves reasoning with environment actions (Yao et al., 2023), while Toolformer trains language models to decide when and how to call APIs through self-supervised data construction (Schick et al., 2023). ToolLLM and Gorilla further scale tool learning to large API collections and evaluate models on tool selection and invocation (Qin et al., 2024, Patil et al., 2024). These works establish tool use as a core interface for LLM agents, but they usually keep individual tool invocations as the basic action unit. We instead consider composing recurring tool sequences into skills and study when a policy should use atomic tools or composite skills.

Experience and Workflow Reuse. Another line of work improves agents by reusing past experience as memory, reflection, or procedural guidance. ExpeL extracts lessons from previous tasks and recalls them during later decision making (Zhao et al., 2024). MemP and Mem0 study procedural or long-term memory for agent experience reuse (Fang et al., 2025, Chhikara et al., 2025). AWM induces workflows from previous web tasks, and AutoManual builds instruction manuals through interaction with environments (Wang et al., 2025, Chen et al., 2024). These methods show that prior trajectories contain reusable knowledge, but context-level reuse does not reduce the number of low-level tool decisions. It also leaves open when a reusable procedure should be executed as an action rather than only used as guidance.

Tool Composition and Skill Construction. Tool composition methods aim to organize multiple tool invocations into higher-level execution units. LLMCompiler plans and executes multiple function calls to improve orchestration efficiency (Kim et al., 2024), while LLM-Tool Compiler and AWO merge or compile recurring tool operations into more efficient interfaces (Singh et al., 2024, Abuzakuk et al., 2026). Recent skill-oriented methods, including SkillRL, SKILL0, and Trace2Skill, further explore how reusable skills can be built or distilled from agent experience (Xia et al., 2026, Lu et al., 2026, Ni et al., 2026). These methods move beyond context-level reuse, but they often focus on constructing or compiling skills, leaving skill-library ambiguity and atomic-versus-skill selection underexplored.

Policy Optimization for Tool-Use Agents. Reinforcement learning has been used to improve multi-turn tool-use agents, where training signals are often defined from final task outcomes. GRPO optimizes policies with group-relative advantages (Shao et al., 2024), and recent work extends outcome-based policy optimization to multi-turn tool-calling agents (Li et al., 2026a, Cai et al., 2026). Other methods introduce step-level credit assignment for multi-step agent rollouts (Wang et al., 2026, Chen et al., 2025, Dong et al., 2025, Yu et al., 2025, Qian et al., 2025). These methods improve learning from long interactions, but they usually assume a fixed action granularity. CIPO instead targets the granularity decision between atomic tools and composite skills.

3. Method

3.1. Problem Formulation

We consider a tool-use task with an input instruction x . At step t , the agent observes the interaction history $h_t = (x, a_1, o_1, \dots, a_{t-1}, o_{t-1})$, where a_i is a selected action and o_i is the observation returned by the environment. The policy π_θ samples the next action as $a_t \sim \pi_\theta(\cdot \mid h_t)$. A rollout $\tau = (x, a_1, o_1, \dots, a_T, o_T)$ ends when the agent produces a final answer or reaches a maximum interaction length, and receives a task reward $R_{\text{task}}(\tau)$.

Let $\mathcal{A}_{\text{atom}}$ denote the set of atomic tools. Each atomic tool corresponds to a single executable operation, such as retrieving a file, reading a document, running a command, or querying an API. We also define a set of composite skills $\mathcal{A}_{\text{skill}}$. Each skill $s \in \mathcal{A}_{\text{skill}}$ is exposed to the policy as one callable action, while its internal execution follows a sequence of atomic tools $s = (u_1, \dots, u_m)$, where $u_i \in \mathcal{A}_{\text{atom}}$. The augmented action space is

$$\mathcal{A} = \mathcal{A}_{\text{atom}} \cup \mathcal{A}_{\text{skill}}. \quad (1)$$

This augmented action space creates a granularity choice. An atomic tool provides fine control through a single operation, whereas a composite skill provides a higher level action that executes a reusable procedure. We denote the granularity of an action by $g(a) \in \{\text{atom}, \text{skill}\}$. A granularity comparison is available at a state h_t when the same history supports both a feasible atomic action and a related composite skill. In this

case, the two actions provide alternative ways to advance the same local subtask at different granularities.

Adaptive tool granularity selection asks the policy to decide, under the current history, whether to take a single atomic step or invoke a composite skill that executes several atomic steps. This is not a global preference for one type of action. An atomic tool is useful when the agent needs intermediate observations or fine control, whereas a skill is useful when its internal procedure matches the current subtask. The policy therefore needs evidence about which granularity works better from the same decision context. CIPO provides this evidence by constructing executable skills and training the policy with counterfactual comparisons between feasible atomic and skill choices.

3.2. Budget-constrained Skill Mining

Mining reusable procedures from tool trajectories requires explicit constraints, because frequent-subsequence mining can easily produce redundant candidates. For example, a repeated routine such as `search_file` $\rightarrow$ `read_file` $\rightarrow$ `extract_answer` may yield both its full sequence and shorter overlapping subsequences as candidate skills. These variants describe related routines but compete as separate actions, which increases the ambiguity of skill selection. We therefore mine a compact set of recurring patterns under explicit budget constraints.

Given a successful rollout τ , we convert its atomic tool invocations into an abstract token sequence. Each tool invocation is represented by its tool name and parameter-key signature, written as $z_t = \text{name}(a_t)[\text{keys}(a_t)]$. This abstraction preserves the structure of a tool invocation while removing task-specific parameter values. For instance, two calls to the same file-reading tool with different file paths share the same token if they require the same parameter keys. We define $\mathcal{D}_{\text{succ}}$ denote the set of token sequences extracted from successful rollouts.

Inspired by byte-pair encoding in tokenization, which repeatedly merges frequent adjacent symbols, we apply a BPE-style merging procedure to $\mathcal{D}_{\text{succ}}$. At each round, the algorithm counts adjacent token pairs and selects the most frequent valid pair:

$$(a^*, b^*) = \arg \max_{(a,b)} \sum_{z \in \mathcal{D}_{\text{succ}}} \sum_{t=1}^{|z|-1} \mathbb{I}[z_t = a, z_{t+1} = b]. \quad (2)$$

All occurrences of (a^*, b^*) are merged into a new composite token. The merge process is constrained by the maximum number of merge rounds K , the minimum pair frequency f_{min} , and the maximum pattern length L_{max} . These constraints bound the number of generated candidates by K and the length of each retained candidate by L_{max} , preventing the skill library from growing with all frequent subsequences in the corpus. We provide implementation details and the values used in our experiments in Appendix A.

The output of this stage is a compact set of symbolic skill candidates. Each candidate stores its merged token, its atomic tool sequence, its frequency statistics, and its merge history. These candidates identify recurring tool-use structures, but they are not yet executable skills. The next stage converts them into callable actions with explicit parameter interfaces and runtime execution support.

3.3. Executable Skill Instantiation

The mining stage returns symbolic skill candidates, each represented as an ordered sequence of atomic tools. However, a symbolic sequence cannot be directly used by the policy. To make a candidate executable, CIPO registers it as a callable skill with a tool schema, an execution plan, and runtime control fields. For a candidate

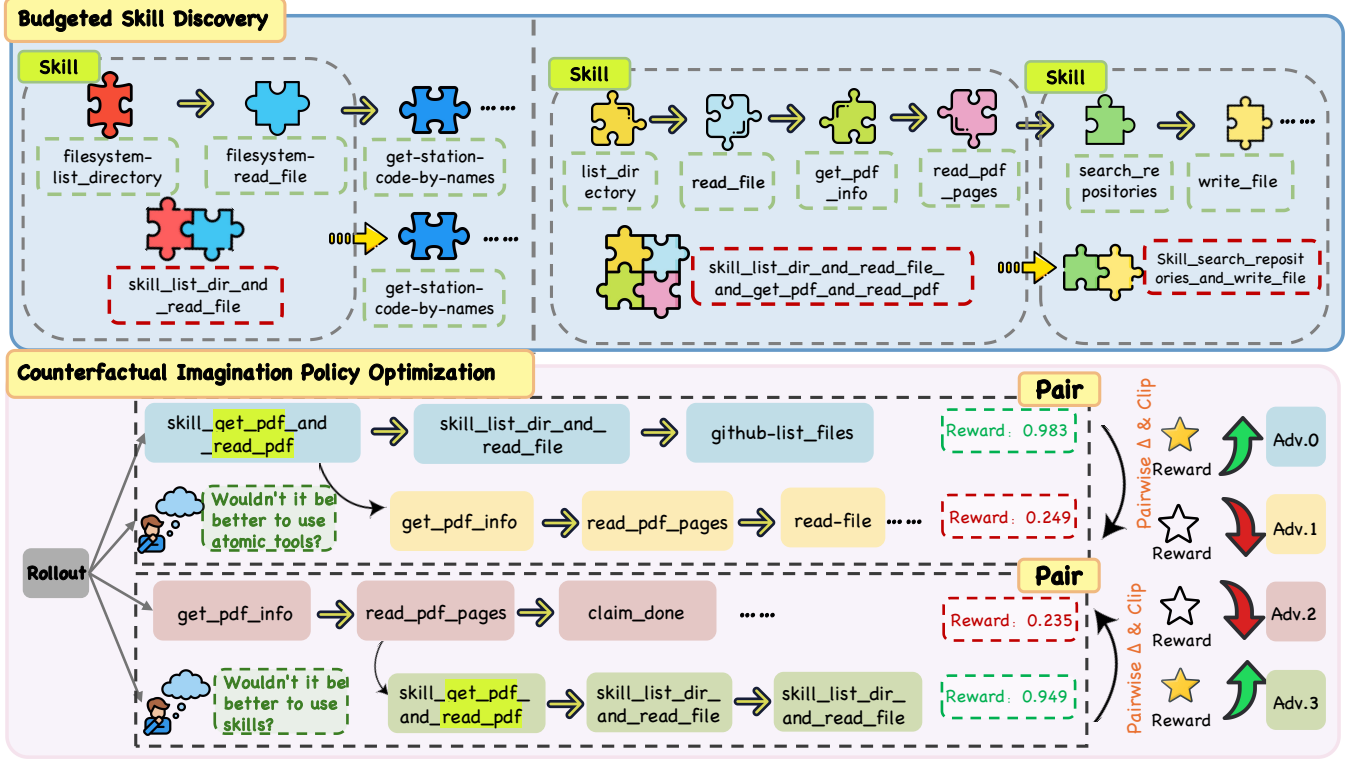


Figure 2: Overview of CIPO. Budgeted skill discovery mines recurring tool-use patterns and instantiates executable skills. Counterfactual imagination policy optimization creates paired branch rollouts by replacing the first eligible granularity choice with an atomic or skill alternative. The clipped reward difference provides a supplementary signal for adaptive tool granularity.

$s = (u_1, \dots, u_m)$, the registered skill exposes a single action to the policy, while internally preserving the ordered execution of its atomic tools.

A central issue is parameter assignment. Some parameters must be provided by the policy because they depend on the current task state, while others can be passed internally from earlier tool outputs. CIPO separates these two cases through parameter analysis. Parameters required by the first atomic tool are exposed by default. For later tools, a parameter is treated as internal if it can be matched to an output field from a previous step; otherwise, it remains exposed to the policy. This design keeps the external skill interface compact while preserving the dataflow needed for execution.

CIPO also assigns each skill an execution template. Most candidates use a sequential template, where atomic tools are executed in order with internal parameter passing. If an intermediate tool returns multiple candidates and a later tool requires a target item, the skill uses a selection template. In this case, the skill exposes a selection control parameter, allowing the policy to decide how to choose from returned candidates. This prevents the skill from becoming a fixed pipeline and keeps the policy involved in runtime decisions.

During execution, CIPO records a trace for every internal step, including the atomic tool name, arguments, returned observation, and execution status. If an internal step fails or cannot be completed, the skill performs a soft interruption. It returns the partial trace and the available intermediate result, then hands control back to the policy. This trace-aware design makes composite skills inspectable and recoverable, rather than opaque actions that either fully succeed or fail without feedback.

3.4. CIPO: Counterfactual Imagination Policy Optimization

After the augmented action space is built, the remaining challenge is to train the policy to choose the appropriate granularity under the current context. Applying GRPO in the augmented action space provides rollout-level rewards but does not directly compare atomic and skill actions from the same decision state. CIPO adds such a comparison by augmenting GRPO with counterfactual branch rollouts. The original task reward remains the main optimization signal, while the counterfactual branch provides a supplementary signal for granularity selection.

Base rollout generation. For each training instruction x , CIPO samples base rollouts from the current policy over the augmented action space $\mathcal{A}$. Each base rollout is a complete interaction that may contain both atomic tool invocations and composite skill calls. To match the rollout budget of GRPO + Skills, CIPO fixes the total number of rollouts per instruction to G . Let G_{base} denote the number of sampled base rollouts and G_{cf} denote the number of counterfactual branches. The rollout budget satisfies

$$G_{\text{base}} + G_{\text{cf}} = G. \quad (3)$$

In our implementation, we use $G = 4$: CIPO first samples two base rollouts and then constructs at most one counterfactual branch for each base rollout. If a base rollout does not contain a decision state where an alternative granularity is available, its reserved branch budget is filled with an additional base rollout. These additional base rollouts participate in the standard GRPO group normalization, but only valid base-branch pairs receive the granularity-specific supplementary reward.

For each base rollout, CIPO stores the interaction history together with the selected actions, action positions, and skill execution traces. This information allows a counterfactual branch to restart from the exact prefix before a granularity decision. Given a base rollout $\tau = (x, a_1, o_1, \dots, a_T, o_T)$, CIPO scans its action sequence and identifies the first decision state where an alternative action at the other granularity is available.

Counterfactual branching. After a base rollout is generated, CIPO constructs a paired counterfactual rollout when an alternative granularity is available. The algorithm scans the base rollout from the beginning and branches at the first decision state where the chosen action can be replaced by a feasible action at the other granularity. If the base action is an atomic tool a_t , CIPO selects a composite skill s^* whose internal tool chain contains a_t . If the base action is a skill $s = (u_1, \dots, u_m)$, CIPO uses its first internal atomic tool u_1 as the alternative action. We denote the alternative action by $\tilde{a}_t$. Detailed replacement rules and examples are provided in Appendix D.

The counterfactual branch starts from the same prefix h_t as the base rollout. CIPO executes $\tilde{a}_t$, appends the returned observation to the history, and lets the same policy complete the remaining interaction. The resulting branch is denoted as τ_{cf} , while the original rollout is denoted as τ_{base} . This construction compares two granularity choices from the same decision context and evaluates their effect through the subsequent trajectory. CIPO branches at most once for each base rollout, which keeps the paired comparison clean and avoids an exponential growth of counterfactual trajectories.

We branch only at the first such decision in each base rollout. This design keeps the base rollout and the branch identical before the replacement and changes only one granularity choice, making the paired reward difference easier to interpret. It also avoids the additional rollout cost and mixed interventions that would arise from branching at every eligible step. Across training, different prompts and sampled rollouts expose different branch points.

Counterfactual supplementary reward. The branch comparison provides a direct signal for the granularity decision. Let $R_0(\tau)$ denote the base rollout reward computed from task progress, skill behavior, and execution efficiency. The detailed reward composition is described in Appendix E. For a valid base-branch pair, CIPO computes the outcome difference

$$\Delta = R_0(\tau_{\text{cf}}) - R_0(\tau_{\text{base}}). \quad (4)$$

A positive Δ means that the alternative granularity leads to a better rollout, while a negative Δ means that the original choice is better. To prevent this signal from dominating the task reward, CIPO scales and clips the difference:

$$r_{\text{cf}} = \text{clip}(\alpha\Delta, -\epsilon_{\text{cf}}, \epsilon_{\text{cf}}), \quad (5)$$

where α controls the strength of the counterfactual signal and ϵ_{cf} sets its maximum magnitude.

CIPO first applies this signal symmetrically to the paired rollouts:

$$\hat{R}(\tau_{\text{cf}}) = R_0(\tau_{\text{cf}}) + r_{\text{cf}}, \quad \hat{R}(\tau_{\text{base}}) = R_0(\tau_{\text{base}}) - r_{\text{cf}}. \quad (6)$$

This reward adjustment encourages the policy to prefer the granularity that performs better under the matched decision context. The counterfactual term is used only as a bounded supplementary signal. The original task reward remains the main signal for policy optimization.

Group-relative policy update. After reward adjustment, CIPO groups all rollouts for the same instruction, including base rollouts, counterfactual branches, and additional base rollouts used to fill the fixed budget. All adjusted rewards are clipped to be non-negative, $\hat{R}_i \leftarrow \max\{0, \hat{R}_i\}$, then normalized within the group:

$$\hat{A}_i = \frac{\hat{R}_i - \text{mean}(\{\hat{R}_j\}_{j=1}^G)}{\text{std}(\{\hat{R}_j\}_{j=1}^G) + \eta}. \quad (7)$$

CIPO then optimizes the policy with the standard GRPO objective using $\hat{A}_i$ as the rollout-level advantage. The core GRPO update remains unchanged; CIPO adds matched counterfactual branches to the rollout group and a bounded granularity-specific signal to the reward.

At inference time, the model observes both atomic tools and composite skills and directly selects actions from $\mathcal{A}$ without counterfactual branching. These branches are used only during training to improve granularity selection. Algorithm 1 summarizes the training procedure.

Overall, CIPO mines recurring patterns from successful trajectories, instantiates them as executable skills, and combines these skills with atomic tools to form the augmented action space. The model is first supervised fine-tuned on successful trajectories in this space to learn the tool-call format and both action granularities. It is then optimized with counterfactual branch rollouts, where the task reward remains the main signal and paired outcome differences provide bounded supplementary signals for granularity selection. At inference time, the policy directly selects actions from the augmented space without counterfactual branching.

4. Experiments

4.1. Experimental Setup

Benchmarks and models. We evaluate CIPO on TOOLATHLON (Li et al., 2025), TOUCAN (Xu et al., 2025b), and TRAJECT-Bench (He et al., 2025), which cover long-horizon tool-use tasks involving retrieval,

file access, structured processing, and API-style operations. We use two open-source backbones, Qwen2.5-7B (Hui et al., 2024) and Llama3.1-8B (Grattafiori et al., 2024), to test whether the method generalizes across model families.

Baselines. We compare CIPO with four groups of baselines: general tool-use methods, including ReAct (Yao et al., 2023) and ToolLLM (Qin et al., 2024); memory-based methods, including ExpeL (Zhao et al., 2024), MemP (Fang et al., 2025), and Mem0 (Chhikara et al., 2025); workflow reuse methods, including AWM (Wang et al., 2025); and tool composition or skill-based methods, including AWO (Abuzakuk et al., 2026), LLMCompiler (Kim et al., 2024), LLM-Tool Compiler (Singh et al., 2024), SkillRL (Xia et al., 2026), SKILL0 (Lu et al., 2026), and Trace2Skill (Ni et al., 2026). We also compare with SFT (Ouyang et al., 2022), standard GRPO (Shao et al., 2024), and “GRPO + Skills” in the ablation study.

Metrics. Tool F1 measures the order-aware match between predicted and reference tool trajectories after tool-name normalization. TSR is the mean over five independent LLM-judge evaluations of 100 episodes per setting, counting both yes and partial as success. Decision efficiency is measured by Comp./Raw, the ratio of compressed policy decisions to expanded atomic calls. A smaller ratio indicates stronger trajectory compression.

4.2. Main Results

Table 1: Main results across three benchmarks with Qwen2.5-7B and Llama3.1-8B. We report Tool F1 (%) and Task Success Rate (TSR, %). TSR is averaged over five independent judge evaluations of the same 100 evaluation episodes in each setting. Avg.Comp./Raw is the average compressed decision ratio for composite-action methods; lower values indicate stronger compression.

Method	TOOLATHLON				TOUCAN				TRAJECT-Bench				Avg. Comp./Raw↓
	Qwen-7B		Llama-8B		Qwen-7B		Llama-8B		Qwen-7B		Llama-8B		
	F1↑	TSR↑	F1↑	TSR↑	F1↑	TSR↑	F1↑	TSR↑	F1↑	TSR↑	F1↑	TSR↑	
General tool-use agents													
ReAct	33.00	22.73	43.79	25.41	42.74	32.68	44.00	34.22	43.64	30.96	33.71	23.58	–
ToolLLM	21.10	11.62	35.43	20.43	26.28	17.59	33.34	27.11	50.10	31.50	54.68	32.00	–
Memory and workflow reuse													
ExpeL	18.87	3.50	30.25	6.20	4.28	0.97	2.88	0.65	6.85	0.50	7.02	1.75	–
MemP	19.96	2.45	21.87	3.78	26.29	2.20	25.20	3.05	36.96	11.05	46.37	15.65	–
Mem0	20.28	12.75	36.73	18.86	31.11	20.35	44.49	28.76	40.50	29.17	49.53	30.32	–
AWM	15.14	6.15	22.93	8.66	4.11	1.05	7.14	0.44	6.54	1.85	12.55	3.06	–
Tool composition and skill construction													
AWO	24.87	15.96	31.45	19.72	21.62	20.34	24.18	18.05	27.86	17.41	30.23	15.68	0.86
LLMComp.	26.92	10.64	21.37	11.28	20.84	12.53	23.16	11.76	18.45	9.12	27.38	15.04	0.93
LLM-ToolComp.	18.38	11.74	20.69	14.83	25.47	16.62	28.11	15.38	23.92	11.67	32.06	18.42	0.89
SkillRL	19.08	10.50	22.60	14.17	27.75	20.15	29.15	17.41	0.82	0.34	37.27	21.36	0.88
SKILL0	18.23	10.74	17.65	9.17	23.77	16.40	24.44	13.50	31.32	18.78	31.43	21.33	1.00
Trace2Skill	12.49	7.37	13.09	6.71	23.83	15.29	24.64	13.89	31.15	20.05	34.45	18.77	0.96
CIPO (Ours)	35.17	30.78	48.32	40.55	48.55	35.50	50.51	41.32	50.99	32.00	55.86	38.75	0.845 ± 0.03

Comparison across method families. Table 1 shows that CIPO achieves the strongest overall performance across all three benchmarks and two backbones. General tool-use agents can invoke external tools, but they

still rely on long atomic trajectories. Memory and workflow reuse methods bring useful prior experience, yet their gains remain unstable because reuse is mainly provided as context or guidance. Tool composition and skill construction methods reduce repeated operations, but they do not explicitly train the policy to decide when a skill should replace an atomic tool. CIPO directly targets this granularity choice through counterfactual branch rollouts, achieving the best Tool F1 and TSR in all six benchmark-backbone settings. This consistent improvement suggests that adaptive granularity training helps the policy match useful tool trajectories, complete tasks more reliably, and transfer across different tool-use environments.

Decision compression. CIPO also gives the best decision compression among methods with composite actions. Its Avg.Comp./Raw is 0.845 ± 0.03 , corresponding to about a 15.5% reduction in policy-level decisions after skill expansion. This ratio is lower than existing composition and skill-based baselines, indicating stronger trajectory compression. More importantly, CIPO improves compression while maintaining the strongest overall TSR and competitive Tool F1. This indicates that CIPO does not simply invoke more skills, but learns when composite skills can reduce repeated decisions without hurting task completion. Full metrics and complete results are provided in Appendix J, with TSR judge details in Appendix L.

4.3. Skill Usage and Granularity Selection

We further examine how skill usage changes during training on TOOLATHLON with Qwen2.5-7B. Figure 3 reports the skill usage ratio in online training rollouts, defined as the fraction of policy-level decisions that invoke composite skills. We include a coverage-derived reference level. Given skill-library coverage c and frequency-weighted average skill length m , the reference ratio is

$$\text{SkillRatio}_{\text{ref}} = \frac{c/m}{c/m + (1 - c)}. \quad (8)$$

This reference estimates the expected skill usage ratio when replaceable atomic steps are compressed by available skills. It provides a library-level reference and should not be interpreted as a state-level optimal policy. We provide the derivation in Appendix G.

GRPO + Skills remains close to the SFT baseline, while CIPO increases skill usage toward the coverage-derived reference. This trend reflects training-time skill frequency rather than state-level granularity correctness. Skill-use statistics on final evaluation trajectories come from a different rollout distribution and therefore need not match Figure 3. Table 2 provides complementary evidence: CIPO improves Tool F1, TSR, and decision compression over GRPO + Skills. Together, these results associate altered skill-use behavior with better task performance and efficiency.

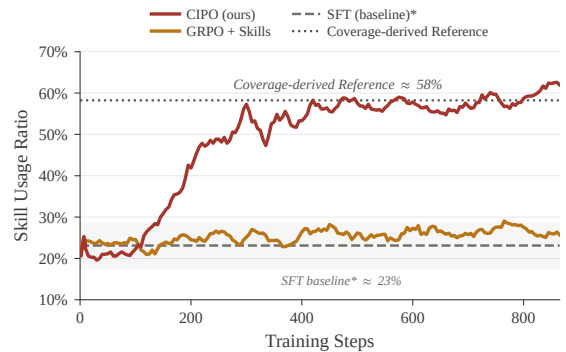


Figure 3: Skill usage during training. CIPO approaches the coverage-derived reference, while GRPO + Skills remains near its initial level.

4.4. Ablation Studies

Table 2 reports ablations averaged across all benchmarks and backbones. Standard GRPO provides an atomic-only baseline with its own SFT initialization. In the augmented action space, GRPO + Skills improves F1, TSR, and decision compression over the reported SFT baseline. CIPO further improves all three metrics, supporting the benefit of counterfactual granularity training beyond access to skills. Removing the budget constraint from skill mining weakens performance relative to full CIPO, highlighting the importance of a compact and controlled skill library.

Table 2: Ablation results averaged over all benchmarks and backbones. $\Delta F1$ and ΔTSR are computed relative to the reported skill-enabled SFT baseline. C/R denotes Comp./Raw, and Decision red. denotes $1 - C/R$.

Method	Variant role	F1 $\uparrow$	$\Delta F1$	TSR $\uparrow$	ΔTSR	C/R $\downarrow$	Decision red. $\uparrow$
SFT	skill-enabled imitation	36.05	0.00	22.91	0.00	0.958	4.2%
Standard GRPO	atomic-only RL	38.92	+2.87	26.74	+3.83	1.000	0.0%
GRPO + Skills	skill-enabled RL	41.20	+5.15	29.30	+6.39	0.912	8.8%
w/o Budgeted Mining	removes mining budget	42.82	+6.77	31.75	+8.84	0.872	12.8%
CIPO-random	random eligible branch	46.72	+10.67	34.90	+11.99	0.858	14.2%
CIPO-first	first eligible branch	48.23	+12.18	36.48	+13.57	0.845	15.5%

We further compare CIPO-first with CIPO-random under the same rollout budget and GRPO group construction, where each instruction uses $G = 4$ rollouts with two base rollouts and up to two counterfactual branches. CIPO-random samples a branch point from all eligible granularity decisions in each base rollout, while CIPO-first branches at the first eligible decision. CIPO-first gives the best average F1, TSR, and decision reduction, suggesting that early granularity decisions provide more compute-efficient counterfactual signal. Full ablations, including budgeted mining, executable instantiation, broader branching strategies, and reward sensitivity, are reported in Appendix H.

5. Conclusion

We presented CIPO, a counterfactual imagination policy optimization framework for adaptive tool granularity selection in LLM agents. CIPO combines budget-constrained construction of executable skills from successful tool-use trajectories with counterfactual branch rollouts, using paired outcome differences from the same decision context to guide the choice between atomic tools and composite skills. Across three benchmarks and two backbones, CIPO improves tool trajectory quality, task success, and decision compression. These results highlight the importance of learning when to invoke composite skills and when to retain atomic control for effective and efficient tool use.

References

- Sami Abuzakuk, Anne-Marie Kermarrec, Rishi Sharma, Rasmus Moorits Veski, and Martijn de Vos. Optimizing agentic workflows using meta-tools. *arXiv preprint arXiv:2601.22037*, 2026.
- Guangfeng Cai, Kaibing Yang, Shuo He, Yu Li, Shengtian Yang, Jiaqi Lv, and Lei Feng. Beyond next-observation prediction: Agent-authored world modeling for sequential decision making. *arXiv preprint arXiv:2606.25421*, 2026.
- Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. Fireact: Toward language agent fine-tuning. *arXiv preprint arXiv:2310.05915*, 2023.
- Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. Automanual: Constructing instruction manuals by llm agents via interactive environmental learning. *Proceedings of Advances in Neural Information Processing Systems*, 37:589–631, 2024.
- Wenjie Chen, Wenbin Li, Di Yao, Xuying Meng, Chang Gong, and Jingping Bi. Gtool: Graph enhanced tool planning with large language model. *arXiv preprint arXiv:2508.12725*, 2025.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Guanting Dong, Yifei Chen, Xiaoxi Li, Jiajie Jin, Hongjin Qian, Yutao Zhu, Hangyu Mao, Guorui Zhou, Zhicheng Dou, and Ji-Rong Wen. Tool-star: Empowering llm-brained multi-tool reasoner via reinforcement learning. *arXiv preprint arXiv:2505.16410*, 2025.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. *arXiv preprint arXiv:2508.06433*, 2025.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *Proceedings of International conference on machine learning*, pages 10764–10799, 2023.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Pengfei He, Zhenwei Dai, Bing He, Hui Liu, Xianfeng Tang, Hanqing Lu, Juanhui Li, Jiayuan Ding, Subhabrata Mukherjee, Suhan Wang, et al. Traject-bench: A trajectory-aware benchmark for evaluating agentic tool use. *arXiv preprint arXiv:2510.04550*, 2025.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. Sok: Agentic skills—beyond tool use in llm agents. *arXiv preprint arXiv:2602.20867*, 2026.
- Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM compiler for parallel function calling. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, Proceedings of Machine Learning Research, pages 24370–24391, 2024.

-
- Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, Xiaochen Wang, Rui Ge, Yuxuan Cao, Yuzhen Huang, Wei Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. *arXiv preprint arXiv:2510.25726*, 2025.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, 2023.
- Xidong Li, Mingyu Guo, Chenchao Xu, Bailin Li, Wenjing Zhu, Yangang Zou, Rui Chen, and Zehuan Wang. Mtdrive: Multi-turn interactive reinforcement learning for autonomous driving. *arXiv preprint arXiv:2601.22930*, 2026a.
- Yu Li, Shenyu Zhang, Rui Wu, Xiutian Huang, Yongrui Chen, Wenhao Xu, Guilin Qi, and Dehai Min. Mateval: A multi-agent discussion framework for advancing open-ended text evaluation. In *Database Systems for Advanced Applications - 29th International Conference, DASFAA*, pages 415–426, 2024a.
- Yu Li, Guangfeng Cai, Shengtian Yang, Han Luo, Shuo Han, Xu He, Dong Li, and Lei Feng. Phgpo: Pheromone-guided policy optimization for long-horizon tool planning. *arXiv preprint arXiv:2602.13691*, 2026b.
- Yu Li, Zhe Yang, Yi Huang, Xin Liu, and Guilin Qi. C³tg: Conflict-aware, composite, and collaborative controlled text generation. In *Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI*, pages 31834–31842, 2026c.
- Zelong Li, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. Autoflow: Automated workflow generation for large language model agents. *arXiv preprint arXiv:2407.12821*, 2024b.
- Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *Proceedings of 2023 IEEE International conference on robotics and automation (ICRA)*, pages 9493–9500. IEEE, 2023.
- Grace Liu, Yuxiao Qu, Jeff Schneider, Aarti Singh, and Aviral Kumar. Cart: Teaching llm agents to know when they know enough. *arXiv preprint arXiv:2510.08517*, 2025.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, et al. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. In *Proceedings of Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183, 2025.
- Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Skill0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*, 2026.
- Dehai Min, Giovanni Vaccarino, Huiyi Chen, Yongliang Wu, Gal Yona, and Lu Cheng. Stop when reasoning converges: Semantic-preserving early exit for reasoning models. *arXiv preprint arXiv:2605.17672*, 2026a.

-
- Dehai Min, Kailin Zhang, Tongtong Wu, and Lu Cheng. Quco-rag: Quantifying uncertainty from the pre-training corpus for dynamic retrieval-augmented generation. In *Findings of the Association for Computational Linguistics, ACL*, pages 16482–16500. Association for Computational Linguistics, 2026b.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Xiaoxi Jiang, and Guanjun Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.
- Kangyun Ning, Yisong Su, Xueqiang Lv, Yuanzhe Zhang, Jian Liu, Kang Liu, and Jinan Xu. Wtu-eval: A whether-or-not tool usage evaluation benchmark for large language models. *arXiv preprint arXiv:2407.12823*, 2024.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Proceedings of Advances in Neural Information Processing Systems*, 2022.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Proceedings of Advances in Neural Information Processing Systems*, pages 126544–126565, 2024.
- Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *Proceedings of The Twelfth International Conference on Learning Representations*, 2024.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Proceedings of Advances in neural information processing systems*, pages 68539–68551, 2023.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

-
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Proceedings of Advances in neural information processing systems*, 36:8634–8652, 2023.
- Simranjit Singh, Andreas Karatzas, Michael Fore, Iraklis Anagnostopoulos, and Dimitrios Stamoulis. An llm-tool compiler for fused parallel function calling. *arXiv preprint arXiv:2405.17438*, 2024.
- Richard S Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2):181–211, 1999.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Tao Wang, Suhang Zheng, and Xiaoxiao Xu. Rtmc: Step-level credit assignment via rollout trees. *arXiv preprint arXiv:2604.11037*, 2026.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of International Conference on Machine Learning*, pages 63897–63911, 2025.
- Zhiheng Xi, Chenyang Liao, Guanyu Li, Zhihao Zhang, Wenxiang Chen, Binghai Wang, Senjie Jin, Yuhao Zhou, Jian Guan, Wei Wu, Tao Ji, Tao Gui, Qi Zhang, and Xuanjing Huang. Agentprm: Process reward models for LLM agents via step-wise promise and progress. In *Proceedings of the ACM Web Conference 2026, WWW*, pages 4184–4195, 2026.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025a.
- Zhangchen Xu, Adriana Meza Soria, Shawn Tan, Anurag Roy, Ashish Sunil Agrawal, Radha Poovendran, and Rameswar Panda. Toucan: Synthesizing 1.5 m tool-agentic data from real-world mcp environments. *arXiv preprint arXiv:2510.01179*, 2025b.
- Kaibing Yang, Guangfeng Cai, Shengtian Yang, Shuo He, Yu Li, Mengyi Liu, Pengwei Chen, Jun Xu, and Lei Feng. Progress-conditioned group policy optimization for long-horizon agentic tasks. *arXiv preprint arXiv:2607.22724*, 2026a.
- Shengtian Yang, Ziyu Xiong, Yu Li, Yewen Li, Qingpeng Cai, and Lei Feng. Srpo: Setwise relative policy optimization for multi-agent llms. *arXiv preprint arXiv:2609.08452*, 2026b.
- Shengtian Yang, Ziyu Xiong, Kaibing Yang, Guangfeng Cai, Yewen Li, Peng Jiang, Gai Kun, Qingpeng Cai, and Lei Feng. Sca: Spatial credit assignment for reinforcement learning of gui agents. *arXiv preprint arXiv:2609.36939*, 2026c.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *Proceedings of The Eleventh International Conference on Learning Representations*, 2023.

-
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- Yuanqing Yu, Zhefan Wang, Weizhi Ma, Shuai Wang, Chuhan Wu, Zhiqiang Guo, and Min Zhang. Steptool: Enhancing multi-step tool usage in llms via step-grained reinforcement learning. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 3952–3962, 2025.
- Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. Agenttuning: Enabling generalized agent abilities for llms. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3053–3077, 2024.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation. In *Proceedings of The Thirteenth International Conference on Learning Representations*, 2025.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *Proceedings of The 12th International Conference on Learning Representations*, 2024.

A. Details of Budget-constrained Skill Mining

This appendix provides additional details for the budget-constrained skill mining procedure described in Section 3.2. The goal of this stage is not to enumerate all frequent subsequences. Instead, the algorithm builds a compact set of recurring tool-use patterns under explicit constraints, which keeps the augmented action space manageable.

A.1. Input Corpus and Token Abstraction

The mining stage uses successful rollouts as its input corpus. For each successful rollout, we extract its atomic tool-call sequence and convert each tool invocation into an abstract token. A tool invocation with tool name u_t and argument dictionary p_t is represented as

$$z_t = u_t[\text{keys}(p_t)], \quad (9)$$

where $\text{keys}(p_t)$ denotes the sorted set of argument keys. This abstraction removes task-specific argument values while preserving the functional structure of the call. For example, two calls to the same file-reading tool with different file paths are mapped to the same token if both calls require the same argument key. This design reduces spurious variants caused by concrete parameter values and makes repeated tool structures easier to identify.

Let $\mathcal{D}_{\text{succ}} = \{z^{(i)}\}_{i=1}^M$ denote the resulting corpus of token sequences, where each $z^{(i)} = (z_1^{(i)}, \dots, z_{T_i}^{(i)})$ is extracted from one successful rollout. Sequences with fewer than two tool invocations are ignored because they cannot produce a composite pattern.

A.2. BPE-style Iterative Merging

The mining procedure follows the idea of byte-pair encoding, which repeatedly merges frequent adjacent symbols. In our setting, the symbols are tool-call tokens rather than text tokens. At each merge round, the algorithm counts adjacent token pairs over the current corpus and selects the most frequent valid pair:

$$(a^*, b^*) = \arg \max_{(a,b)} \sum_{z \in \mathcal{D}_{\text{succ}}} \sum_{t=1}^{|z|-1} \mathbb{I}[z_t = a, z_{t+1} = b]. \quad (10)$$

The selected pair (a^*, b^*) is replaced by a new composite token $a^* \oplus b^*$ at all of its occurrences. The corpus is then updated, and the next round is performed on the updated token sequences. Because composite tokens can be merged again in later rounds, the procedure can discover tool patterns longer than two calls while still relying only on adjacent pair statistics at each step.

The merge history records the selected pair, the new composite token, the pair frequency, the merge round, the resulting atomic length, and the compression gain. The atomic length of a composite token is computed recursively from its merge history. This allows the algorithm to enforce a maximum pattern length even when a candidate is built through several merge rounds.

A.3. Budget Constraints

The procedure is controlled by three main constraints. First, the maximum number of merge rounds is K . Second, a selected pair must appear at least $f_{\min}$ times in the current corpus. Third, the atomic length of the

Table 3: Default hyperparameters for budget-constrained skill mining.

Hyperparameter	Meaning	Default value
K	maximum merge rounds	50
$f_{\min}$	minimum adjacent-pair frequency	3
$L_{\max}$	maximum atomic length of a candidate	6
$L_{\min}$	minimum atomic length of a candidate	2
$\rho_{\min}$	minimum usage ratio for pruning	0.01

resulting composite token must not exceed $L_{\max}$. If the most frequent pair violates the length constraint, the algorithm searches the remaining pairs in descending frequency order and selects the highest-frequency pair that satisfies the same constraints. If no valid pair remains, the merge process stops.

After merging, the algorithm applies a pruning step. Each composite token is counted in the final updated corpus. A candidate is removed if its usage is too low, measured by its current frequency and usage ratio, or if its decomposed atomic sequence is shorter than the minimum macro length. This pruning step removes candidates that were useful during intermediate compression but no longer occur often enough after later merges.

In our experiments, we use $K = 50$, $f_{\min} = 3$, and $L_{\max} = 6$ unless otherwise stated. We also use a minimum macro length of 2 and a usage-based pruning threshold. These values were chosen to keep the skill library compact while preserving frequent local routines. We list the implementation values used in our experiments in Table 3.

A.4. Size and Length Bounds

We now state the simple bounds induced by the budget constraints. These bounds are not intended as an optimality guarantee for BPE. They only formalize why the mining stage cannot grow without control.

Proposition 1. Let K be the maximum number of merge rounds. The number of composite tokens generated by the mining procedure is at most K . After pruning, the number of retained symbolic skill candidates is also at most K .

Proof. Each merge round creates at most one new composite token. Since the procedure performs no more than K merge rounds, it can generate at most K composite tokens before pruning. The pruning stage only removes candidates and never creates new ones. Therefore, the number of retained candidates is no larger than the number of generated composite tokens, which is at most K .

Proposition 2. Let $L_{\max}$ be the maximum allowed atomic length of a composite token. Every retained skill candidate has atomic length at most $L_{\max}$.

Proof. Before a pair is merged, the algorithm computes the atomic length of the resulting composite token. A merge is accepted only if this length is no larger than $L_{\max}$. Since every generated composite token is produced by an accepted merge, every generated candidate satisfies the length constraint. The pruning stage only removes candidates, so every retained candidate also satisfies the same bound.

These two propositions explain the role of the budget constraints. The number of candidate skills is bounded by K , while the internal length of each candidate is bounded by $L_{\max}$. As a result, the skill library cannot grow with all frequent subsequences in the trajectory corpus.

A.5. Computational Complexity

Let $N = \sum_{i=1}^M |z^{(i)}|$ be the total number of tokens in the initial corpus. At each round, the algorithm scans the current corpus to count adjacent pairs and then scans it again to replace the selected pair. Since merging only shortens the current corpus, each round costs $O(N)$ under a straightforward implementation. With at most K rounds, the overall mining complexity is $O(KN)$, excluding minor overhead from candidate validation and semantic naming. The memory cost is $O(N + K)$, where $O(N)$ stores the current corpus and $O(K)$ stores the merge history.

The length check can be implemented by caching the atomic length of each token when it is created. With this cache, validating a candidate merge takes constant time after the pair is selected. Our implementation computes lengths from the merge history, which is sufficient for the scale of our experiments. A cached implementation would have the same selected candidates and a lower constant factor.

A.6. Semantic Skill Naming and Output

Each retained candidate is decomposed into its atomic tool sequence. The tool names are then used to construct a readable skill identifier. For example, a candidate containing a directory listing tool followed by a file reading tool can be named as a list-and-read style skill. The output of the mining stage stores the skill identifier, merged token, atomic token sequence, tool-name sequence, original frequency, current frequency, usage ratio, merge round, and compression gain.

The output of the mining stage is a symbolic set of reusable tool-use structures. To register these structures as skills, we first decompose each retained composite token into its atomic tool sequence using the merge history. We then assign a skill identifier, attach the corresponding tool schemas, and store the frequency statistics and merge metadata. The registered skill record serves as the input to skill instantiation, where the symbolic sequence is converted into a callable tool with explicit parameters, internal dataflow, selection control, execution traces, and soft interruption.

B. Details of Executable Skill Instantiation

This appendix describes how a symbolic skill candidate is converted into a callable tool. The mining stage only returns an ordered sequence of atomic tools. Skill instantiation adds a tool schema, parameter interface, execution plan, runtime dataflow, selection control, execution traces, and soft interruption. This conversion is necessary because a frequent tool sequence is not automatically executable under a new task state.

Skill registration. For each retained symbolic candidate $s = (u_1, \dots, u_m)$, we register a new skill tool and add it to the augmented tool library together with the original atomic tools. Each registered skill contains the following fields:

- `skill_id`, `name`, and `description`: the identifier and natural-language description of the skill;
- `tool_chain`: the ordered sequence of internal atomic tools;

-
- `parameters_schema`: the arguments exposed to the policy;
 - `execution_plan`: the execution order and argument source for each internal tool;
 - `pipe_map`: the mapping from earlier outputs to later inputs;
 - `trace_enabled`: whether internal execution traces are recorded.

The registered record is used by the skill interpreter to execute the symbolic sequence as a single callable action.

Parameter exposure and internal piping. A symbolic tool sequence usually contains two types of parameters. Some parameters must be provided by the policy because they depend on the current instruction or state. Other parameters can be derived from earlier tool outputs. We separate these cases during registration:

$$\begin{aligned} P_{\text{exp}}(s) &= \{p \mid p \text{ is supplied by the policy}\}, \\ P_{\text{int}}(s) &= \{p \mid p \text{ is resolved from previous outputs}\}. \end{aligned} \tag{11}$$

For the first atomic tool u_1 , required parameters are exposed to the policy by default. For each later tool u_i , a parameter is treated as internal if it can be matched to an output field from a previous step. Otherwise, it remains exposed to the policy. This rule keeps the external skill call compact while preserving the dataflow needed for multi-step execution.

Execution templates. Each skill is assigned an execution template. The default template is sequential. Under this template, internal tools are executed in order, and later arguments are either supplied by the policy or piped from previous outputs. Some skills require a selection step. This case appears when an earlier tool returns multiple candidates and a later tool requires one target item, identifier, path, or index. For these candidates, the skill exposes a `select_strategy` parameter. In our implementation, `select_strategy` supports strategies such as `rank-0`, `rank-1`, `random`, and `filter`. During execution, if an intermediate output is a list, the strategy determines which item is selected before extracting the required field for the next tool.

Runtime execution and soft interruption. At runtime, the policy invokes a skill by producing a single tool invocation with the skill name and exposed parameters. The interpreter then executes the internal plan. For each internal step, it resolves arguments from exposed parameters and piped values from earlier outputs. The interpreter records the tool name, arguments, returned observation, and execution status for every internal step.

If an internal step cannot continue, the skill does not fail as an opaque black box. It returns the partial trace, the intermediate outputs that have already been obtained, the interruption step, and the interruption reason. Control is then returned to the policy. The policy may inspect the partial result and continue with atomic tools. For example, if a directory-listing step returns an empty file list, the following file-reading step cannot choose a valid file path. The skill then stops after the listing step, records that no candidate file was available, and returns this partial trace to the policy. If the file list is not empty but the selected file cannot be read, the skill stops at the reading step and returns the selected path together with the failure status. In both cases, the policy can continue from the returned state rather than losing all intermediate information.

Example 1: list-and-read skill. Consider a mined candidate

$$s_{\text{read}} = (\text{list_directory}, \text{read_file}). \quad (12)$$

A fixed pipeline would not know which file should be read after the directory is listed. The registered skill therefore exposes the directory path and a selection strategy:

$$\text{skill_list_and_read}(\text{path}, \text{select_strategy}). \quad (13)$$

Its execution plan is:

$$\begin{aligned} 1. & \text{list_directory}(\text{path}) \rightarrow \text{files}, \\ 2. & \text{read_file}(\text{path} = \text{select}(\text{files}, \text{select_strategy})). \end{aligned} \quad (14)$$

Here, the first path is supplied by the policy. The second path is not supplied directly. It is selected from the file list returned by the first step. If no file can be selected, the skill stops before calling `read_file` and returns the partial trace.

Example 2: metadata-to-content skill. Consider another mined candidate

$$s_{\text{pdf}} = (\text{get_pdf_info}, \text{read_pdf_pages}). \quad (15)$$

This sequence first inspects a PDF and then reads its content. The registered skill can expose the PDF path and the page range:

$$\text{skill_inspect_and_read_pdf}(\text{pdf_path}, \text{page_range}). \quad (16)$$

Its execution plan is:

$$\begin{aligned} 1. & \text{get_pdf_info}(\text{pdf_path}) \rightarrow \text{metadata}, \\ 2. & \text{read_pdf_pages}(\text{pdf_path}, \text{page_range}). \end{aligned} \quad (17)$$

In this example, the policy must provide `pdf_path`, because the skill needs to know which document to inspect. The later call to `read_pdf_pages` also requires a page range. If the relevant pages can be inferred from the metadata returned by the first tool, `page_range` can be filled internally through the pipe map. Otherwise, it must be exposed as a policy-controlled parameter. In our example, the metadata only describes the document structure and does not determine which pages are relevant to the current task. Therefore, `page_range` remains an exposed parameter. This example shows that skill instantiation does not simply pass the first tool’s arguments to all later tools. Instead, it specifies which parameters are supplied by the policy and which parameters can be resolved from intermediate outputs.

Skill instantiation turns a symbolic subsequence into an executable action. It exposes parameters that require policy control, pipes parameters that can be resolved from intermediate outputs, supports selection when an internal step returns multiple candidates, and records execution traces for recovery. This design allows composite skills to reduce repeated decisions without becoming fixed and opaque pipelines.

C. CIPO Training Algorithm

Algorithm 1 summarizes the training procedure of CIPO. For each instruction, CIPO keeps the total rollout budget fixed at $G = 4$, which matches the GRPO + Skills baseline. It first samples two base rollouts, then

attempts to construct at most one counterfactual branch for each base rollout. If fewer than two branches can be constructed, CIPO samples additional base rollouts until the group contains four rollouts. Only valid base-branch pairs receive the granularity-specific supplementary reward. All four rollouts are included in the group-relative policy update.

Algorithm 1 CIPO Training with Fixed Rollout Budget

Require: Policy π_θ , old policy $\pi_{\theta_{\text{old}}}$, augmented action space $\mathcal{A}$, instruction x , rollout budget $G = 4$, base rollout count $G_{\text{base}} = 2$, base reward R_0

```

1: Sample two base rollouts  $\mathcal{B} = \{\tau_{\text{base}}^{(1)}, \tau_{\text{base}}^{(2)}\}$  from  $\pi_\theta$ 
2: Initialize rollout group  $\mathcal{G} \leftarrow \mathcal{B}$  and pair set  $\mathcal{P} \leftarrow \emptyset$ 
3: for each base rollout  $\tau_{\text{base}} \in \mathcal{B}$  do
4:   Scan the actions in  $\tau_{\text{base}}$  from the beginning
5:   Find the first state  $h_t$  with a feasible alternative action  $\tilde{a}_t$  at the other granularity
6:   if such  $h_t$  and  $\tilde{a}_t$  exist then
7:     Fork from the prefix before  $a_t$ 
8:     Execute  $\tilde{a}_t$ , then continue the interaction with  $\pi_\theta$ 
9:     Obtain counterfactual branch  $\tau_{\text{cf}}$ 
10:    Add  $\tau_{\text{cf}}$  to  $\mathcal{G}$ 
11:    Add  $(\tau_{\text{base}}, \tau_{\text{cf}})$  to  $\mathcal{P}$ 
12:   end if
13: end for
14: while  $|\mathcal{G}| < G$  do
15:   Sample an additional base rollout  $\tau_{\text{extra}}$  from  $\pi_\theta$ 
16:   Add  $\tau_{\text{extra}}$  to  $\mathcal{G}$ 
17: end while
18: for each rollout  $\tau_i \in \mathcal{G}$  do
19:   Compute base reward  $R_0(\tau_i)$ 
20:   Initialize adjusted reward  $\hat{R}(\tau_i) \leftarrow R_0(\tau_i)$ 
21: end for
22: for each valid pair  $(\tau_{\text{base}}, \tau_{\text{cf}}) \in \mathcal{P}$  do
23:    $\Delta \leftarrow R_0(\tau_{\text{cf}}) - R_0(\tau_{\text{base}})$ 
24:    $r_{\text{cf}} \leftarrow \text{clip}(\alpha\Delta, -\epsilon_{\text{cf}}, \epsilon_{\text{cf}})$ 
25:    $\hat{R}(\tau_{\text{cf}}) \leftarrow \hat{R}(\tau_{\text{cf}}) + r_{\text{cf}}$ 
26:    $\hat{R}(\tau_{\text{base}}) \leftarrow \hat{R}(\tau_{\text{base}}) - r_{\text{cf}}$ 
27: end for
28: Clip adjusted rewards to be non-negative
29: Normalize  $\{\hat{R}(\tau_i)\}_{i=1}^G$  within the group to obtain advantages  $\{\hat{A}_i\}_{i=1}^G$ 
30: Update  $\pi_\theta$  with the standard GRPO objective

```

D. Counterfactual Replacement Rules

This appendix describes how CIPO constructs the alternative action used in a counterfactual branch. The branch does not evaluate a tool output in isolation. Instead, it changes the action granularity at one decision state, executes the alternative action from the same prefix, and lets the same policy complete the remaining interaction.

Scanning rule. For each base rollout, CIPO scans the action sequence from the beginning. It creates a branch at the first action where a feasible alternative at the other granularity exists. The prefix before this action is preserved. The alternative action is executed from the same history, and the policy continues from the modified observation.

Skill-to-atomic replacement. If the chosen action is a composite skill $s = (u_1, \dots, u_m)$, the counterfactual action is the first internal atomic tool u_1 . The arguments for u_1 are obtained from the original skill call through the execution plan. In particular, exposed skill parameters are mapped to the required arguments of the first internal tool whenever the mapping is defined. If the required arguments cannot be resolved, no branch is constructed at this action.

Atomic-to-skill replacement. If the chosen action is an atomic tool a , CIPO searches the skill library for a skill whose internal tool chain contains a . The candidate set is

$$\mathcal{C}(a) = \{s \in \mathcal{A}_{\text{skill}} \mid a \in \text{chain}(s)\}. \quad (18)$$

A candidate is feasible if its exposed parameters can be constructed from the original atomic arguments and the current history. When multiple candidates are feasible, CIPO selects the shortest feasible skill, with frequency statistics used as a tie breaker. If no feasible skill exists, no counterfactual branch is created at this action.

Argument mapping. The replacement action must be meaningful under the same decision context. For a skill-to-atomic replacement, CIPO maps the skill arguments to the schema of the first internal atomic tool. For an atomic-to-skill replacement, CIPO maps the original atomic arguments to the exposed parameters of the selected skill. Parameters that cannot be copied directly may be resolved from the current history or from the skill execution plan. If the mapping fails, the candidate is rejected. This rule prevents CIPO from creating branches with empty or irrelevant arguments.

Example A: Atomic PDF reading to PDF inspection skill

Base action. The base rollout calls an atomic tool:

```
read_pdf_pages(pdf_file_path, start_page, end_page).
```

Candidate skill. The skill library contains:

```
skill_get_pdf_and_read_pdf_pages
```

with internal chain:

```
get_pdf_info → read_pdf_pages.
```

Replacement. The skill chain contains `read_pdf_pages`, so CIPO can replace the atomic call with this skill if the exposed parameters can be filled. The original atomic arguments provide `pdf_file_path`, `start_page`, and `end_page`, which are sufficient for the skill call.

Counterfactual branch. The branch first executes `get_pdf_info`, then executes `read_pdf_pages` with the same page range. This branch tests whether adding metadata inspection before reading improves the later trajectory.

Example B: PDF inspection skill to first atomic tool

Base action. The base rollout calls:

```
skill_get_pdf_and_read_pdf_pages(pdf_file_path, start_page, end_page).
```

Internal chain. The skill executes:

```
get_pdf_info → read_pdf_pages.
```

Replacement. For a skill-to-atomic branch, CIPO uses the first internal atomic tool:

```
 $\tilde{a}$  = get_pdf_info(pdf_file_path).
```

The argument `pdf_file_path` is copied from the original skill call through the execution plan.

Counterfactual branch. The branch only performs metadata inspection at the branch point. The policy then decides whether to read pages, call another tool, or revise its plan. This branch tests whether preserving control after the first atomic step is better than executing the full skill immediately.

Example C: Atomic file reading to list-and-read skill

Base action. The base rollout calls:

```
read_file(path="/workspace/report.txt").
```

Candidate skill. The skill library contains:

```
skill_list_and_read(path, select_strategy)
```

with internal chain:

```
list_directory → read_file.
```

Feasibility check. The skill contains `read_file`, but it also requires a directory path and a selection strategy. If the directory can be derived from the original file path, CIPO maps `/workspace/report.txt` to `/workspace`. The selection strategy can be filled by a default strategy or by a policy-controlled parameter.

Counterfactual branch. When the mapping is feasible, the branch lists the directory, selects a file according to `select_strategy`, and then reads it. This branch tests whether a broader list-and-read routine helps, or whether the original direct file read was more appropriate.

Example D: Write-and-run skill to atomic file writing

Base action. The base rollout calls:

```
skill_write_file_and_run_cmd(content, path, cmd).
```

Internal chain. The skill executes:

```
write_file → run_cmd.
```

Replacement. The counterfactual action is the first internal atomic tool:

```
 $\tilde{a}$  = write_file(content, path).
```

Counterfactual branch. After writing the file, the policy regains control. It may run the command, inspect the file, modify the script, or use another tool. This branch tests whether executing the full write-and-run routine is better than preserving control after file creation.

Example E: Infeasible atomic-to-skill replacement

Base action. The base rollout calls:

`update_cells(spreadsheet_id, range, values).`

Candidate skill. A candidate skill contains:

`read_sheet → update_cells.`

Feasibility check. Although the skill chain contains `update_cells`, the skill may require a sheet name, a read range, or a selection rule that cannot be recovered from the original update call or the current history. In this case, the replacement would not be executable from the same decision context.

Decision. CIPO rejects this candidate and continues scanning later actions in the base rollout. This example shows that CIPO does not replace every atomic tool with any skill that contains it. The alternative action must be executable with meaningful arguments.

E. Reward Composition

This appendix describes the reward used during CIPO training. The reward has two layers. First, each rollout receives a base reward R_0 from task progress, skill behavior, and execution efficiency. Second, valid base-branch pairs receive a bounded counterfactual adjustment that provides feedback for tool granularity selection.

Base reward. Before counterfactual adjustment, each rollout receives

$$R_0(\tau) = R_{\text{task}}(\tau) + R_{\text{skill}}(\tau) + R_{\text{eff}}(\tau). \quad (19)$$

The task component measures whether the rollout covers the reference tools. The skill component rewards useful skill calls and penalizes irrelevant or repeated skill calls. The efficiency component rewards shorter decision trajectories only when the rollout has sufficient task progress.

Task component. R_{task} is computed from coverage of the reference tool sequence. Tool names are normalized before matching. Exact matches receive full credit, substring matches receive partial credit, and token-overlap matches receive credit when their Jaccard score is sufficiently high. The coverage score is combined with an order-preservation bonus, so a rollout receives higher task reward when it covers the correct tools in a more consistent order. If the rollout produces no tool invocations, it receives a zero-step penalty.

Skill component. R_{skill} is an additive term. A skill receives positive credit only when its internal tool chain overlaps with the reference tools. A fully successful relevant skill receives the largest bonus, while a partially

successful relevant skill receives a smaller bonus according to the fraction of successful internal steps. If a skill has no overlap with the reference tools, it receives a penalty. Repeated calls to the same skill are also penalized. The total skill bonus is clipped to avoid a collapse in which the policy calls skills only to obtain reward.

Efficiency component. R_{eff} encourages shorter decision trajectories, but only when the rollout has sufficient task progress. In our implementation, the efficiency bonus is gated by task completion and task coverage. A compression bonus is added when the rollout uses skills to reduce the number of policy decisions. Very short rollouts with low task coverage receive an under-exploration penalty. This design prevents the policy from receiving high reward for stopping early without solving the task.

Counterfactual adjustment. For each valid base-branch pair, CIPO computes the base-reward difference

$$\Delta = R_0(\tau_{\text{cf}}) - R_0(\tau_{\text{base}}). \quad (20)$$

The difference is converted into a bounded supplementary reward:

$$r_{\text{cf}} = \text{clip}(\alpha\Delta, -\epsilon_{\text{cf}}, \epsilon_{\text{cf}}). \quad (21)$$

CIPO applies this signal symmetrically to the paired rollouts:

$$\hat{R}(\tau_{\text{cf}}) = R_0(\tau_{\text{cf}}) + r_{\text{cf}}, \quad \hat{R}(\tau_{\text{base}}) = R_0(\tau_{\text{base}}) - r_{\text{cf}}. \quad (22)$$

Rollouts without a valid counterfactual pair retain their base reward before clipping. After all paired adjustments, we apply $\hat{R}(\tau_i) \leftarrow \max\{0, \hat{R}(\tau_i)\}$ to every rollout before group-relative normalization.

Group-relative normalization. For each instruction, CIPO forms a group of $G = 4$ rollouts. The group contains two base rollouts, valid counterfactual branches, and additional base rollouts used to fill the fixed budget when fewer branches are available. The adjusted reward of rollout i is normalized within the group:

$$\hat{A}_i = \frac{\hat{R}_i - \text{mean}(\{\hat{R}_j\}_{j=1}^G)}{\text{std}(\{\hat{R}_j\}_{j=1}^G) + \eta}. \quad (23)$$

CIPO then applies the standard GRPO objective. For generated token t in rollout i , define

$$\rho_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t} \mid h_{i,t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid h_{i,t})}. \quad (24)$$

The policy objective is

$$\begin{aligned} \mathcal{J}_{\text{CIPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} & \left[\min\left(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A}_i\right) \right. \\ & \left. - \beta D_{\text{KL}}[\pi_{\theta}(\cdot \mid h_{i,t}) \parallel \pi_{\text{ref}}(\cdot \mid h_{i,t})] \right]. \end{aligned} \quad (25)$$

CIPO therefore preserves the standard GRPO optimization form. Its difference lies in how the rollout group is constructed and how the reward is adjusted before group-relative normalization.

Role of the counterfactual term. The counterfactual reward is a bounded supplementary signal, not a standalone causal estimate. The branch compares two rollouts that share the same prefix but differ in one granularity choice, after which both continue under the current policy. This comparison gives direct feedback about whether the atomic or skill action worked better under the matched decision context. The task reward remains the main optimization signal throughout training.

F. Training Dynamics

We provide the training reward curves of CIPO in Figure 4. The curves are reported for three benchmarks, TOOLATHLON, TOUCAN, and TRAJECT-Bench, with Qwen-7B and Llama-8B as backbone models. Here Qwen-7B and Llama-8B denote Qwen2.5-7B and Llama3.1-8B, respectively. The logged reward is the group-level training reward used during policy optimization, which includes task progress, skill behavior, efficiency, and the bounded counterfactual adjustment described in Appendix E. These curves are used to analyze the optimization process. The final evaluation results are reported separately in Table 1.

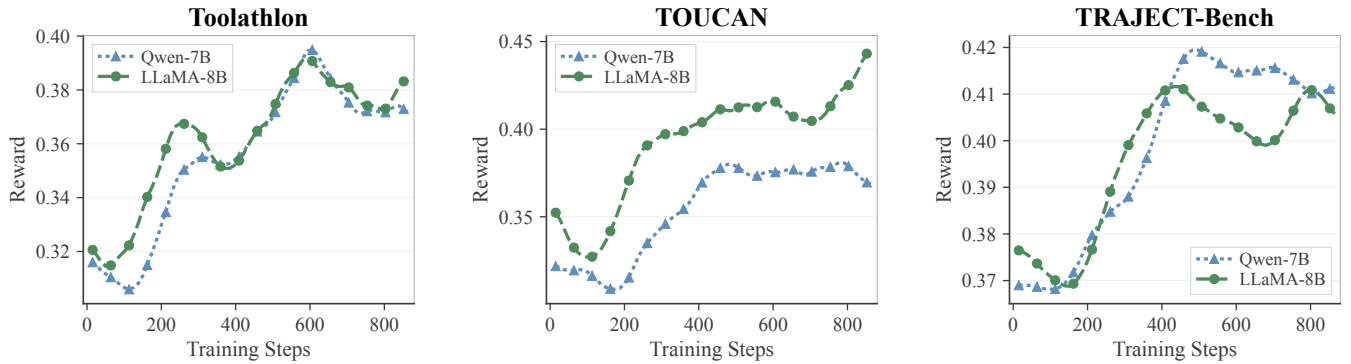


Figure 4: Training reward curves of CIPO across three benchmarks and two backbone models. The reward generally increases during training across TOOLATHLON, TOUCAN, and TRAJECT-Bench, suggesting that the counterfactual branch signal can be optimized together with the task reward.

Overall, CIPO shows a consistent upward trend across the evaluated settings. On TOOLATHLON, both backbones improve after the early training stage and reach substantially higher rewards than their initial values. The curves contain local fluctuations, but the overall direction remains positive, indicating that the policy gradually learns to use the augmented action space more effectively. On TOUCAN, the improvement is especially clear for Llama-8B, whose reward increases steadily throughout training. Qwen-7B also improves and then enters a relatively stable region, suggesting that the optimization signal remains useful after the policy has learned many common tool-use patterns. On TRAJECT-Bench, both models improve rapidly in the early and middle stages. The reward later fluctuates around a higher plateau, which is expected because this benchmark contains more diverse trajectories and the sampled rollouts may vary in difficulty across updates.

The reward curves are not expected to be strictly monotonic. CIPO samples rollouts online, and each update can contain different task instances, base rollouts, and counterfactual branches. In addition, the counterfactual branch may expose a better or worse granularity choice under the current policy, which introduces controlled variation in the group rewards. These fluctuations are useful during training because they provide contrastive feedback for deciding between atomic tools and composite skills. The important observation is that the rewards increase or stabilize at higher levels across all benchmark-backbone combinations, which supports the stability of the CIPO optimization process.

These dynamics also help interpret the main results. The improvement is not limited to a single benchmark or a single backbone, suggesting that the method does not rely on a benchmark-specific skill library or a model-specific behavior. Instead, CIPO provides a general training signal for adaptive tool granularity selection. The main evaluation in Table 1 further shows that this training trend translates into better Tool F1, TSR, and decision compression.

G. Derivation of the Coverage-derived Skill Usage Reference

This appendix explains how we compute the coverage-derived reference used in Figure 3. This reference is not a learned policy. It is an estimate of the expected skill-call ratio when the atomic steps covered by the mined skill library are compressed into composite skill calls.

Let an atomic-only trajectory contain N atomic tool invocations. Suppose the mined skill library can cover a fraction c of these atomic calls. Then cN atomic calls are replaceable by skills, while the remaining $(1 - c)N$ atomic calls are not covered by the skill library and must remain as atomic decisions.

Let m denote the frequency-weighted average length of the mined skills. If the covered atomic calls are compressed by skills of average length m , then the cN covered atomic calls are replaced by approximately

$$\frac{cN}{m} \quad (26)$$

skill calls. The uncovered part still requires

$$(1 - c)N \quad (27)$$

atomic tool invocations. Therefore, after compression, the expected number of policy-level decisions is

$$\frac{cN}{m} + (1 - c)N, \quad (28)$$

where the number of skill decisions is cN/m . The expected skill usage ratio is therefore

$$\text{SkillRatio}_{\text{ref}} = \frac{cN/m}{cN/m + (1 - c)N} = \frac{c/m}{c/m + (1 - c)}. \quad (29)$$

This quantity reflects the expected fraction of policy-level decisions that would be skill calls if all replaceable atomic steps were compressed by available skills. The coverage c increases the reference ratio because more atomic steps can be replaced by skills. The average length m has the opposite effect on the number of skill calls because a longer skill replaces more atomic calls with a single decision. The uncovered fraction $1 - c$ remains as atomic decisions and reduces the expected skill-call ratio.

We use this value as a coverage-derived reference level in Figure 3. It should not be interpreted as a state-level optimal policy. Instead, it provides a library-level reference for judging whether a trained policy uses skills at a plausible rate given the amount of reusable structure available in the skill library. The reference in Figure 3 is computed for the TOOLATHLON skill library used with Qwen2.5-7B and is used only to analyze training-time behavior. Figure 3 measures skill usage in online training rollouts, whereas Table 12 reports skill usage on the final evaluation trajectories. The two therefore characterize different rollout distributions, and their absolute skill-usage ratios are not expected to match.

Table 4: Detailed ablation results across all benchmark-backbone settings. Each entry reports F1 / TSR. Avg. reports the arithmetic mean over the six settings.

Method	TOOLATHLON		TOUCAN		TRAJECT-Bench		Avg. F1	Avg. TSR
	Qwen-7B	Llama-8B	Qwen-7B	Llama-8B	Qwen-7B	Llama-8B		
SFT	25.88 / 15.24	39.42 / 23.58	34.56 / 19.72	36.71 / 22.84	37.04 / 24.30	42.69 / 31.78	36.05	22.91
Standard GRPO	28.90 / 18.42	42.10 / 27.96	38.25 / 24.70	40.68 / 27.65	40.20 / 27.80	43.39 / 33.91	38.92	26.74
GRPO + Skills	31.10 / 22.20	44.35 / 32.55	40.20 / 30.10	42.30 / 31.20	42.85 / 29.10	46.40 / 30.65	41.20	29.30
w/o Budgeted Mining	31.40 / 24.20	45.10 / 34.20	42.90 / 31.50	44.60 / 33.20	44.80 / 30.10	48.10 / 37.30	42.82	31.75
w/o Executable Instantiation	31.50 / 23.70	44.80 / 33.60	41.80 / 30.70	43.70 / 32.50	43.60 / 29.80	47.20 / 36.00	42.10	31.05
w/o CF Reward	32.84 / 25.63	46.72 / 36.40	42.80 / 31.70	44.95 / 33.80	44.60 / 29.80	47.59 / 32.57	43.25	31.65
CIPO-late	33.90 / 28.50	47.20 / 37.90	45.60 / 33.40	47.25 / 38.20	48.10 / 30.20	52.51 / 36.40	45.76	34.10
CIPO-random	34.28 / 28.96	47.65 / 38.82	46.20 / 34.00	48.65 / 39.35	48.61 / 30.72	54.93 / 37.55	46.72	34.90
CIPO-all	34.86 / 29.10	49.05 / 39.80	47.20 / 33.40	49.72 / 39.20	48.40 / 30.65	56.39 / 37.85	47.60	35.00
CIPO-first	35.17 / 30.78	48.32 / 40.55	48.55 / 35.50	50.51 / 41.32	50.99 / 32.00	55.86 / 38.75	48.23	36.48

Table 5: Detailed decision compression results for ablation variants. C/R denotes Comp./Raw. Lower values indicate stronger decision compression.

Method	TOOLATHLON		TOUCAN		TRAJECT-Bench		Avg. C/R
	Qwen-7B	Llama-8B	Qwen-7B	Llama-8B	Qwen-7B	Llama-8B	
SFT	0.960	0.940	0.950	0.930	1.000	0.970	0.958
Standard GRPO	1.000	1.000	1.000	1.000	1.000	1.000	1.000
GRPO + Skills	0.920	0.880	0.910	0.860	0.960	0.942	0.912
w/o Budgeted Mining	0.880	0.820	0.860	0.790	0.950	0.932	0.872
w/o Executable Instantiation	0.873	0.815	0.846	0.781	0.960	0.921	0.866
w/o CF Reward	0.891	0.842	0.873	0.809	0.958	0.931	0.884
CIPO-late	0.875	0.805	0.870	0.835	0.935	0.846	0.861
CIPO-random	0.906	0.800	0.860	0.825	0.925	0.832	0.858
CIPO-all	0.830	0.765	0.820	0.775	0.960	0.950	0.850
CIPO-first	0.855	0.772	0.860	0.812	0.910	0.861	0.845

H. Full Ablation Results

This appendix provides the full ablation results across all benchmark-backbone settings. The main paper reports averaged ablation results for readability. Here, we expand each ablation into six settings: TOOLATHLON with Qwen-7B and Llama-8B, TOUCAN with Qwen-7B and Llama-8B, and TRAJECT-Bench with Qwen-7B and Llama-8B. C/R denotes Comp./Raw, where lower values indicate stronger decision compression.

Component effects. Standard GRPO provides an atomic-only comparison with a separate SFT initialization. In the augmented action space, GRPO + Skills improves F1, TSR, and decision compression over the reported SFT baseline. CIPO further improves these metrics. Removing the counterfactual reward while retaining branch rollouts weakens performance, supporting the contribution of the paired outcome difference to granularity training.

Skill construction and instantiation. Removing budgeted mining reduces performance because the skill library becomes more redundant and harder to select from. Removing executable instantiation also hurts performance, showing that symbolic subsequences are not enough. Skills need explicit parameter interfaces, internal dataflow, selection control, execution traces, and soft interruption to work reliably in new task states.

Branch-point selection. CIPO-random and CIPO-late keep the same rollout budget and group construction as CIPO-first, using $G = 4$ rollouts with two base rollouts and at most two counterfactual branches. They differ only in how the branch point is selected. CIPO-random samples from all eligible granularity decisions, while CIPO-late prefers later eligible decisions. Both variants are weaker than CIPO-first on average, suggesting that the first eligible decision gives a cleaner comparison under the fixed group budget.

CIPO-all is an extra-compute variant that branches at every eligible granularity decision in each base rollout. Since it no longer keeps the group size fixed, its rollout cost grows with the number of eligible branch points. It improves some individual settings, such as Llama-8B on TOOLATHLON and TRAJECT-Bench, and achieves stronger compression on TOOLATHLON and TOUCAN. However, its average F1 and TSR remain below CIPO-first, and its compression becomes worse on TRAJECT-Bench. This indicates that broader branching is not automatically better once computation and reward noise are considered, supporting first-eligible branching as a more stable and compute-efficient default.

Reward sensitivity. Figure 5 evaluates the sensitivity of CIPO to the counterfactual reward scale α and clipping bound ϵ_{cf} . Each variant changes one hyperparameter while keeping the other settings fixed. Reducing α weakens the granularity signal, while increasing α can overemphasize paired reward differences. Similarly, a narrow clipping range suppresses useful counterfactual differences, whereas a wide clipping range can make the supplementary signal too dominant. The default setting used in the main experiments achieves the highest overall F1 and TSR among these variants, indicating that CIPO benefits from a bounded counterfactual signal that is strong enough to guide granularity selection but not strong enough to override the task reward.

I. Hyperparameters and Training Details

This appendix reports the hyperparameters and implementation details used in our experiments. Qwen-7B and Llama-8B denote Qwen2.5-7B and Llama3.1-8B, respectively. We omit machine-specific paths and private credentials from this description.

Overall pipeline. CIPO follows a five-stage pipeline. First, recurring tool-use patterns are mined from successful trajectories. Second, the retained candidates are instantiated as executable skills and combined with the original atomic tools to form the augmented tool library. Third, the model is supervised fine-tuned on successful trajectories in the augmented action space. Fourth, the policy is optimized with CIPO using counterfactual branch rollouts. Finally, the trained policy is evaluated in the augmented action space without counterfactual branching. The same pipeline is used for both Qwen-7B and Llama-8B.

Skill mining. We use budget-constrained BPE-style mining over successful tool-use trajectories. Each tool invocation is abstracted by its tool name and parameter-key signature, which removes task-specific argument values while preserving tool structure. Table 6 lists the mining hyperparameters.

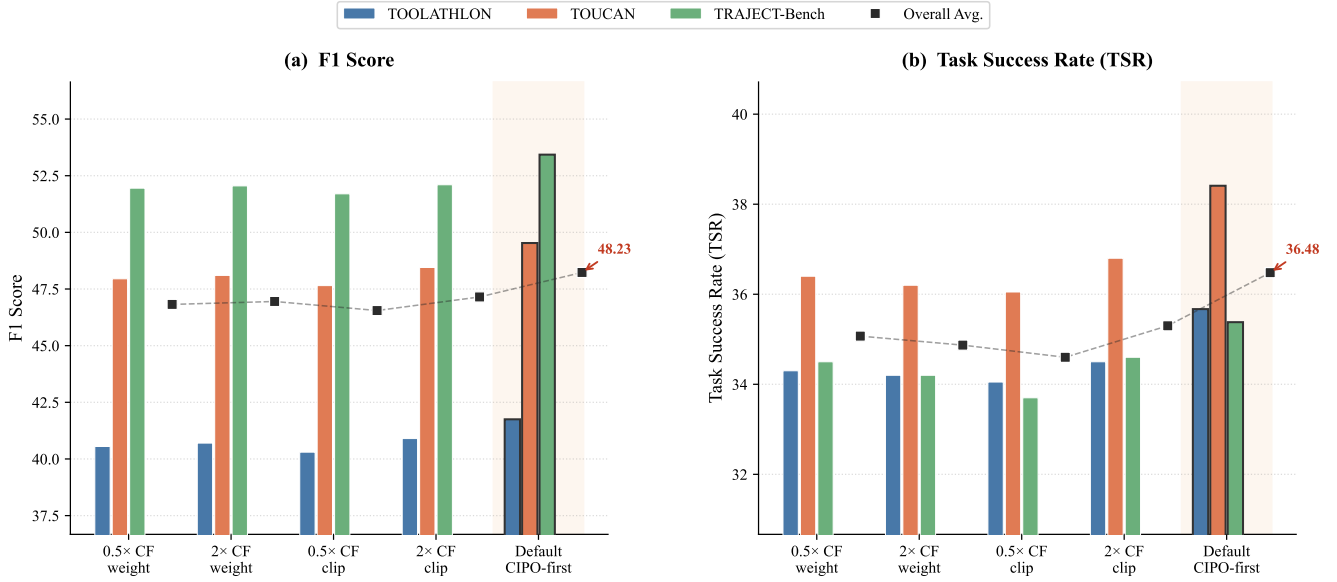


Figure 5: Sensitivity analysis of the counterfactual reward. We vary one hyperparameter at a time around the default setting, including the reward scale α and the clipping bound ϵ_{cf} . Default CIPO-first achieves the best overall balance across benchmarks and models.

Table 6: Hyperparameters for budget-constrained skill mining.

Hyperparameter	Value
Input trajectories	successful trajectories only
Token abstraction	tool name + parameter-key signature
Maximum merge rounds K	50
Minimum pair frequency $f_{\min}$	3
Minimum skill length	2 atomic tools
Maximum skill length $L_{\max}$	6 atomic tools
Minimum usage ratio for pruning	0.01

Skill instantiation. The mined symbolic candidates are converted into executable skills before being exposed to the policy. Each skill is registered with a parameter schema, execution plan, internal dataflow map, execution trace, and interruption behavior. Table 7 summarizes the instantiation settings.

Supervised fine-tuning. The skill-enabled SFT stage initializes the policy in the augmented action space. It teaches the model the tool-call format and exposes it to both atomic tools and composite skills before policy optimization. We use LoRA fine-tuning and compute the loss on assistant tokens only. The SFT hyperparameters are shown in Table 8. Standard GRPO uses a separate atomic-only SFT initialization, as detailed in Appendix O.

CIPO policy optimization. CIPO starts from the SFT checkpoint and optimizes the policy with group-relative policy optimization. In the paper implementation, we keep the rollout budget fixed at $G = 4$ for each instruction. Each group contains two base rollouts, up to two counterfactual branches, and additional

Table 7: Skill instantiation settings.

Item	Value
Execution templates	sequential, select, conditional
Selection strategies	rank-0, rank-1, random, filter
Trace recording	enabled
Soft interruption	enabled
Skill interface	exposed parameters + internal piping

Table 8: Skill-enabled supervised fine-tuning hyperparameters.

Hyperparameter	Value
Training data	successful trajectories in augmented action space
Epochs	3
Learning rate	1×10^{-5}
Per-device batch size	2
Gradient accumulation steps	8
Maximum sequence length	4096
Warmup ratio	0.05
LoRA rank	64
LoRA alpha	128
LoRA dropout	0.10
LoRA target modules	q, k, v, o, gate, up, down projections
Weight decay	0.01
Save steps	200
Logging steps	10
Loss mask	assistant tokens only

base rollouts when fewer than two valid branches are available. All four rollouts are used for group-relative normalization, while only valid base-branch pairs receive the counterfactual supplementary reward. Table 9 gives the policy-optimization hyperparameters used for the 7B and 8B backbones.

Reward configuration. The base reward combines task progress, skill behavior, and efficiency terms. The counterfactual branch adds a bounded supplementary signal based on the paired reward difference. Table 10 lists the reward hyperparameters.

Counterfactual branch construction. For each base rollout, CIPO scans the generated actions from the beginning and branches at the first eligible granularity decision. If the chosen action is an atomic tool, the branch replaces it with a feasible skill whose internal chain contains that tool. If the chosen action is a skill, the branch replaces it with the first internal atomic tool of the skill. The branch starts from the same prefix, executes the replacement action, and lets the current policy complete the remaining interaction. This keeps the comparison tied to the same decision context while preserving a fixed group budget.

Large-model training setup. The 7B and 8B experiments use a two-GPU training path. The model is loaded with automatic device mapping, gradient checkpointing is enabled, and LoRA is used to keep the

Table 9: CIPO policy-optimization hyperparameters for Qwen-7B and Llama-8B.

Hyperparameter	Value
Initialization	SFT checkpoint
Action space	atomic tools + executable skills
Epochs	3
Learning rate	2×10^{-6}
Rollout group size G	4
Base rollouts per instruction	2
Counterfactual branches per instruction	up to 2
Fallback for missing branches	additional base rollouts
Branch point	first eligible granularity decision
KL coefficient β	0.05
Sampling temperature	0.7
Per-device batch size	1
Gradient accumulation steps	4
Maximum sequence length	3072
Maximum generation length	384
LoRA rank	32
LoRA alpha	64
LoRA dropout	0.05
LoRA target modules	q, k, v, o, gate, up, down projections
Gradient checkpointing	enabled
Save steps	50
Logging steps	1

number of trainable parameters small. Rollout generation is performed online, and each rollout group is immediately used for reward computation and policy optimization.

Evaluation protocol. At inference time, CIPO directly selects actions from the augmented tool library without counterfactual branching. Evaluation uses deterministic decoding and a fixed execution budget. Table 11 lists the evaluation settings for CIPO.

Metric computation. Tool F1 is computed as an order-aware match between the predicted tool-use trajectory and the reference trajectory after tool-name normalization. TSR is evaluated by an LLM judge, as described in Appendix L. Comp./Raw compares policy-level decisions after skill compression with the expanded atomic calls of the same executed trajectory. Additional metrics and confidence intervals are provided in Appendix J.

J. Additional Results and Uncertainty Estimates

Additional evaluation metrics. Table 12 reports additional CIPO metrics that are not shown in the main table. We include trajectory-quality metrics, skill-use metrics, reliability metrics, and decision-efficiency details. Prec. and Rec. denote tool precision and recall. NTAcc denotes next-tool accuracy, which measures whether the next selected tool matches the reference trajectory at each step. Prog. denotes progress rate, indicating the fraction of episodes with non-zero task progress. Skill Use denotes the fraction of policy-level

Table 10: Reward hyperparameters.

Hyperparameter	Value
Skill reward weight λ_{skill}	0.30
Complete skill reward	1.00
Passed internal-step reward	0.20
Failed internal-step penalty	-0.50
Efficiency reward gate	0.40
Under-exploration penalty	0.05
Under-exploration threshold	3
Counterfactual reward scale α	0.15
Counterfactual clipping bound ϵ_{cf}	0.10

Table 11: CIPO evaluation hyperparameters.

Hyperparameter	Value
Evaluation episodes per setting	100
Maximum turns	30
Maximum expanded atomic calls	50
Inference temperature	0.0
Top-p	0.95
Batch size	1
Continuation mechanism	enabled
Inference action space	atomic tools + executable skills
Counterfactual branches at inference	disabled
Main reported metrics	Tool F1, TSR, Comp./Raw

decisions that invoke composite skills, and Skill Cov. denotes the fraction of available skills used at least once. Comp. and Raw denote compressed policy decisions and expanded atomic calls, respectively. The reported Avg. Comp./Raw is the arithmetic mean of the six setting-level ratios, rather than the ratio between the averaged Comp. and Raw counts.

The additional metrics provide a more detailed view of CIPO’s behavior. The precision, recall, and NTAcc columns decompose the Tool F1 results in Table 1, showing whether gains come from more accurate tool selection, broader tool coverage, or better step-level choices. The skill-related columns show that CIPO uses composite skills across settings without relying on the entire skill library uniformly. Skill Use varies across benchmarks, with lower usage on TRAJECT-Bench, which is consistent with its weaker decision-compression gain. The Comp. and Raw columns further explain the Avg.Comp./Raw value reported in the main table by showing the compressed policy decisions and the corresponding expanded atomic executions.

Evaluation uncertainty. Table 13 reports uncertainty estimates for the main CIPO results. For Tool F1 and Comp./Raw, we estimate 95% confidence intervals by resampling evaluation episodes with replacement and recomputing the corresponding episode-level metric. TSR is judge-based and is independently evaluated five times on the same set of 100 episodes. We report its mean and standard deviation across these five judge runs. The main table reports the mean TSR for readability, while Table 13 additionally shows the variation across repeated evaluations.

Table 12: Additional evaluation metrics of CIPO across benchmarks and backbones.

Benchmark	Backbone	Prec.	Rec.	NTAcc	Prog.	Skill Use	Skill Cov.	Comp.	Raw
TOOLATHLON	Qwen-7B	45.30	28.75	19.72	83.00	16.03	29.17	6.30	7.37
TOOLATHLON	Llama-8B	59.50	40.70	23.67	95.00	29.24	31.25	8.55	11.07
TOUCAN	Qwen-7B	47.80	49.33	47.80	78.00	9.50	27.50	7.30	8.49
TOUCAN	Llama-8B	49.70	51.35	50.20	84.00	22.80	42.30	8.95	11.02
TRAJECT-Bench	Qwen-7B	53.40	48.80	52.15	78.00	9.10	19.15	4.55	5.00
TRAJECT-Bench	Llama-8B	60.05	52.22	46.24	96.00	2.04	21.28	12.25	14.23
Average	–	52.63	45.19	39.96	85.67	14.79	28.44	7.98	9.53

Table 13: Uncertainty estimates for CIPO. Tool F1 and Comp./Raw are reported with episode-level bootstrap 95% confidence interval half-widths. TSR is reported as mean $\pm$ standard deviation across five independent judge evaluations of the same 100 evaluation episodes.

Benchmark	Backbone	F1	TSR	Comp./Raw
TOOLATHLON	Qwen-7B	35.17 $\pm$ 1.42	30.78 $\pm$ 1.65	0.855 $\pm$ 0.024
TOOLATHLON	Llama-8B	48.32 $\pm$ 1.87	40.55 $\pm$ 2.01	0.772 $\pm$ 0.028
TOUCAN	Qwen-7B	48.55 $\pm$ 1.62	35.50 $\pm$ 1.70	0.860 $\pm$ 0.024
TOUCAN	Llama-8B	50.51 $\pm$ 1.70	41.32 $\pm$ 1.85	0.812 $\pm$ 0.027
TRAJECT-Bench	Qwen-7B	50.99 $\pm$ 1.45	32.00 $\pm$ 1.63	0.910 $\pm$ 0.020
TRAJECT-Bench	Llama-8B	55.86 $\pm$ 1.44	38.75 $\pm$ 1.81	0.861 $\pm$ 0.018
Average	–	48.23 $\pm$ 1.15	36.48 $\pm$ 1.35	0.845 $\pm$ 0.030

K. Dataset Statistics

We evaluate CIPO on three tool-use benchmarks that cover complementary sources of difficulty. TOOLATHLON contains diverse long-horizon tasks that require agents to maintain coherent tool-use decisions across many dependent steps. TOUCAN contains tool-agentic trajectories built from real-world MCP environments and emphasizes tool selection from a large API space. TRAJECT-Bench focuses on trajectory-aware tool-use evaluation, where the ordering and coherence of tool invocations are important in addition to the final response. These benchmarks allow us to test whether CIPO improves adaptive tool granularity under long trajectories, large tool pools, and trajectory-sensitive execution.

Table 14: Statistics of the processed benchmark splits used in our experiments.

Benchmark	Avg. tool invocations / episode	# Unique tools	Main difficulty
TOOLATHLON	~ 13.5	1,250	Long-horizon dependency
TOUCAN	~ 3.2	15,294	Large-scale tool selection
TRAJECT-Bench	~ 6.4	715	Trajectory coherence

TOOLATHLON has the longest trajectories among the three benchmarks, with about 13.5 tool invocations per episode. It is well suited for evaluating whether composite skills can reduce repeated low-level decisions while preserving the dependencies between distant tool invocations. This setting is important for CIPO

because long trajectories provide more opportunities for recurring tool-use routines, but also make incorrect granularity choices more costly.

TOUCAN has shorter trajectories on average, but its tool pool is substantially larger. This benchmark tests whether the policy can identify useful tools and skills from a broad action space. For CIPO, TOUCAN is useful for evaluating whether skill construction and counterfactual granularity training remain effective when the main difficulty comes from tool selection rather than trajectory length alone.

TRAJECT-Bench lies between TOOLATHLON and TOUCAN in trajectory length, with about 6.4 tool invocations per episode. Its trajectory-aware evaluation emphasizes whether the generated tool-use path is coherent, ordered, and aligned with the task structure. This makes it useful for testing whether CIPO improves tool-use quality beyond simply increasing skill invocation.

The three benchmarks are complementary. TOOLATHLON stresses long-horizon dependency, TOUCAN stresses large-scale tool selection, and TRAJECT-Bench stresses coherent tool-use trajectories. This combination allows us to evaluate CIPO across different forms of tool-use difficulty and to test whether adaptive tool granularity selection is useful beyond a single benchmark setting.

L. Task Success Evaluation

We use an LLM judge to evaluate task completion because tool-use tasks can often be solved by multiple valid tool trajectories. Exact matching against a single reference trajectory may understate success when the agent uses functionally equivalent tools or follows a different valid order. For each benchmark-backbone-method setting, we evaluate 100 episodes and independently judge the resulting trajectories five times using the same judge model and prompt. The main TSR is the arithmetic mean of the five run-level TSR values.

Judge model and inputs. The judge model is Qwen3.5-plus. We set temperature to 0.1 and maximum generation length to 512 tokens. Each evaluation trajectory is judged independently in five runs. For each episode, the input contains four fields: the user task, the ground-truth tool chain, the agent’s actual tool-call chain, and the agent’s final answer. Each tool invocation is formatted with its step index, tool name, arguments, and tool output. To keep the prompt length controlled, each tool output is truncated to 500 characters, the user instruction is truncated to 1000 characters, and the final response is truncated to 500 characters.

Evaluation principle. The judge evaluates the agent’s tool-use strategy rather than low-level simulator artifacts. In particular, execution errors returned by the simulator, empty outputs, or API-like failures are not treated as agent errors if the selected tool and arguments are reasonable. The reference tool chain is used as one possible solution path, not as the only valid answer. Functionally equivalent tools, minor naming differences, version suffixes, and format differences are considered acceptable when they satisfy the same task requirement. For long tasks with more than five reference tools, partial completion is allowed when the agent covers the main stages of the task.

TSR computation. The judge assigns each episode one of three completion labels: `yes`, `partial`, or `no`. The main tables report Task Success Rate (TSR) as the proportion of valid judgments labeled `yes` or `partial`. This metric includes partial completion in long-horizon tool-use tasks. We separately record the `yes-only` strict success rate, which excludes all episodes labeled `partial` or `no`. To assess whether the

improvement persists under stricter evaluation, we additionally report repeated yes-only judgments and environment-based task verification in Section L.1.

Judge System Prompt

You are a professional evaluator of tool-use trajectories. Your task is to judge whether an AI agent’s tool-call strategy is reasonable and whether it shows a correct understanding of the user task.

Important evaluation principles:

1. Evaluate the tool-use strategy, not simulator artifacts. Tool errors, empty returns, or API failures should not be penalized when the chosen tool type and arguments are reasonable.
2. The ground-truth tool chain is only one possible solution path. Functionally equivalent tools or alternative valid APIs should be accepted.
3. Focus on task decomposition. If the agent correctly identifies the main intent and selects tools in the right direction, it should receive credit even when the exact tool names differ.
4. Match tool names flexibly. Version suffixes, case differences, underscores and hyphens should be treated as equivalent when their functions match.
5. Be tolerant of small models. The evaluated agents are 1.5B to 8B models, and selecting relevant tool types in complex multi-step tasks should receive credit.
6. For complex tasks with more than five reference tools, partial coverage of key stages can be judged as partial completion.
7. Repeated calls should not be penalized by themselves. They may reflect attempts with different parameters.

Aggregation. For each setting, we independently evaluate the same 100 episodes in five judge runs. For judge run r , we aggregate outputs over all valid episodes and compute

$$\text{TSR}^{(r)} = \frac{N_{\text{yes}}^{(r)} + N_{\text{partial}}^{(r)}}{N_{\text{valid}}^{(r)}}. \quad (30)$$

A judge response that cannot be parsed into the required JSON format is marked as an evaluation error and excluded from $N_{\text{valid}}^{(r)}$ for that run. The TSR reported in the main tables is the mean over the five independent judge runs:

$$\text{TSR} = \frac{1}{R} \sum_{r=1}^R \text{TSR}^{(r)}, \quad R = 5. \quad (31)$$

We also report the average judge score, strict success rate, partial rate, and fail rate.

Judge User Prompt Template

User task

{user_prompt}

Reference tool chain

The following ground-truth tool chain is provided as a reference path only, not as the unique correct solution:

{gt_tools}

Agent tool-call chain

{tool_chain_detail}

Agent final response

{final_response}

Please evaluate the agent's tool-use strategy and output only the following JSON object:

```
{
  "task_completed": "yes/partial/no",
  "score": <integer from 0 to 10>,
  "tool_selection_correct": <true/false>,
  "tool_order_reasonable": <true/false>,
  "key_tools_covered": <number of covered key tools>,
  "key_tools_total": <number of key reference steps>,
  "reason": "<brief reason in 1-2 sentences>"
}
```

Scoring guideline:

- 9–10: the tool-use strategy is correct and covers all key steps.
- 7–8: the strategy is correct and covers most key steps with minor omissions.
- 5–6: the agent understands the task and covers major stages.
- 3–4: the agent calls only a few relevant tools and misses core steps.
- 1–2: the tool invocations are mostly unrelated to the task.
- 0: no effective tool invocation is made.

Completion label:

- yes: score ≥ 7 .
- partial: score ≥ 4 and score < 7 .
- no: score < 4 .

L.1. Strict and Execution-Based Task Success

We complement the main TSR with two evaluations to examine whether CIPO's improvement persists when partial labels receive no credit and when task completion is verified directly from the execution environment.

Strict judge-based evaluation. In addition to the repeated Qwen3.5-plus evaluation used for the main TSR, we re-evaluate the same 100 episodes in each of the six benchmark-backbone settings using gpt-4o.

Method identities are hidden from the judge, the temperature is set to 0.1, and each trajectory is independently judged five times. Only the **yes** label, corresponding to a score of at least 7, counts as strict success. For each judge run, we average strict success rates across the six settings, and report the mean and standard deviation over the five runs.

Execution-based evaluation. We additionally apply TOOLATHLON’s original task-specific execution evaluators (Li et al., 2025) to the same 100 TOOLATHLON tasks for Qwen2.5-7B and Llama3.1-8B over three training seeds. These evaluators check whether the resulting environment state satisfies the task requirements, without relying on an LLM judgment of the tool trajectory. Alternative valid tool paths are accepted when they produce the required final state. For each seed, we average the success rates across the two backbones, then report the mean and standard deviation across the three seeds. This execution-based evaluation is restricted to TOOLATHLON; TOUCAN and TRAJECT-Bench are covered by the strict judge-based evaluation above.

Table 15: Strict and execution-based task success (%). The **yes-only** column averages all six benchmark–backbone settings and reports mean $\pm$ standard deviation across five judge runs. The execution-based column covers TOOLATHLON and reports mean $\pm$ standard deviation across three training seeds after averaging the two backbones.

Method	yes-only strict success	Execution-based success
	All six settings	TOOLATHLON
SFT	16.32 $\pm$ 0.42	11.67 $\pm$ 0.58
Standard GRPO	19.63 $\pm$ 0.51	14.83 $\pm$ 0.76
GRPO + Skills	21.40 $\pm$ 0.47	17.67 $\pm$ 0.58
CIPO	27.68 $\pm$ 0.54	25.17 $\pm$ 0.76

Results. Table 15 presents the **yes-only** results. CIPO achieves the highest mean in both columns, exceeding GRPO + Skills by 6.28 percentage points in **yes-only** strict success and 7.50 percentage points in the TOOLATHLON comparison. The strict evaluation shows that CIPO retains an advantage when partial labels are excluded from the success count. The corresponding advantage on TOOLATHLON provides complementary evidence of improved task completion without relying on LLM judgment or partial-credit

M. Effect of the Skill-mining Budget

We further analyze the effect of the BPE merge budget K , which controls the maximum number of merge rounds during budget-constrained skill mining. This hyperparameter directly affects the size and granularity of the skill library. A smaller budget produces a compact library, but it may miss reusable tool-use routines that appear across successful trajectories. A larger budget can recover more candidate routines, but it may also introduce redundant, overlapping, or task-specific skills. These redundant skills increase the action space and make it harder for the policy to decide which composite action should replace atomic tools.

Figure 6 shows the task success rate under different values of K . TSR increases from $K = 10$ to $K = 50$, indicating that a very small budget under-extracts useful recurring patterns. The gain becomes weaker after $K = 50$, and performance declines when K is further increased to 100 and 200. This trend suggests that skill mining benefits from a moderate budget: the library should be large enough to cover common routines, but

not so large that it creates excessive ambiguity. The default value $K = 50$ achieves the best average TSR, which supports the budgeted design used in the main experiments.

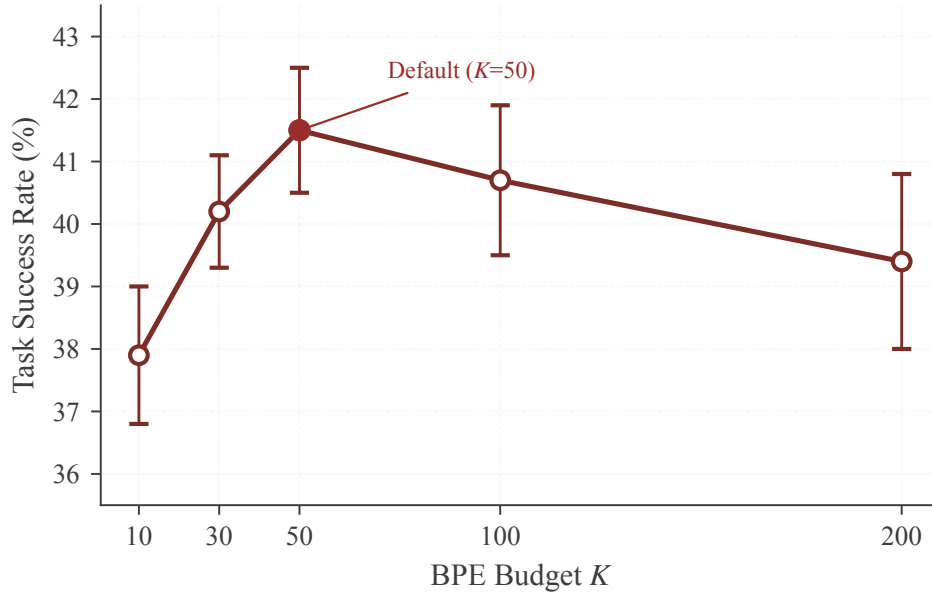


Figure 6: Effect of the BPE merge budget K . TSR improves as more reusable routines are mined, but declines when the skill library becomes too large and redundant. The default value $K = 50$ gives the best average TSR.

N. Task-level Decision Compression

The main table reports Comp./Raw as an aggregate measure of decision compression. To better understand this effect at the task level, we visualize the relation between policy-level decisions and expanded atomic calls for individual evaluation episodes. For each episode, the horizontal axis records the number of decisions made by the policy, where each composite skill counts as one decision. The vertical axis records the number of atomic tool invocations after expanding all executed skills. The diagonal line $y = x$ indicates no compression: every policy decision corresponds to exactly one atomic operation.

Figure 7 compares CIPO with GRPO + Skills in the same augmented action space. GRPO + Skills remains close to the diagonal, indicating limited decision compression. In contrast, CIPO places more episodes above the diagonal, where each policy decision corresponds to more than one executed atomic operation. This means that a smaller number of policy decisions can trigger a larger number of executed atomic operations through composite skills.

This task-level pattern complements the Comp./Raw results in Table 1. The average ratio shows that CIPO reduces policy-level decisions overall, while the scatter plot shows that this reduction is not driven by a few isolated cases. Instead, many episodes exhibit moderate compression, and some longer episodes show larger gaps between decision steps and atomic calls. These results support the claim that CIPO improves efficiency by learning when composite skills can safely replace repeated atomic tool choices.

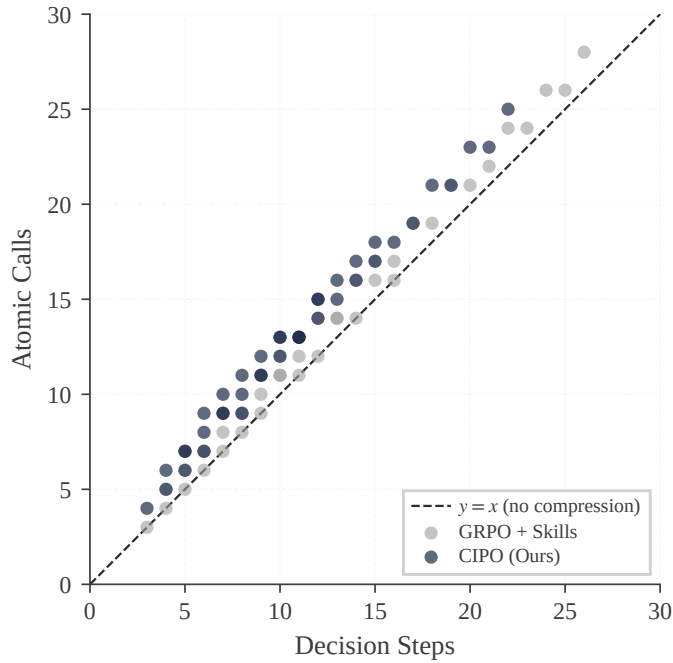


Figure 7: Task-level decision compression. Each point represents one evaluation episode. The diagonal line indicates no compression. Compared with GRPO + Skills, CIPO places more episodes above the diagonal, showing that composite skills allow fewer policy-level decisions to execute more atomic operations.

O. SFT Initializations and GRPO Baselines

Separate SFT initializations. We use separate SFT initializations for atomic-only and skill-enabled policies. Standard GRPO starts from an SFT model trained with atomic tools only. GRPO + Skills starts from a separately trained SFT model with access to both atomic tools and composite skills. CIPO also follows the skill-enabled SFT pipeline described in Section 3.4. The SFT baseline reported in the ablation tables and skill-usage analyses denotes the skill-enabled model.

Action spaces and policy optimization. Standard GRPO uses the atomic action space $\mathcal{A}_{\text{atom}}$ during both reinforcement learning and evaluation. GRPO + Skills uses the augmented action space $\mathcal{A} = \mathcal{A}_{\text{atom}} \cup \mathcal{A}_{\text{skill}}$ and applies the standard GRPO objective without counterfactual branching or paired reward adjustments. CIPO uses the same augmented action space but constructs matched counterfactual branches and adds a bounded supplementary reward based on paired outcome differences.

Interpretation of comparisons. Standard GRPO provides an atomic-only baseline, whereas GRPO + Skills provides the skill-enabled baseline for assessing CIPO beyond access to executable skills. The comparison between Standard GRPO and GRPO + Skills includes both the action-space extension and the corresponding SFT preparation. In the ablation tables, $\Delta F1$ and ΔTSR are measured relative to the reported skill-enabled SFT baseline. For Standard GRPO, these differences compare the final policies and do not measure improvement over its own SFT initialization. Analyses of skill invocation and granularity selection use GRPO + Skills as the GRPO baseline.

Decision compression. For Standard GRPO, each tool decision corresponds to one atomic call. Comp./Raw therefore equals 1 for trajectories with at least one tool call, and the corresponding decision reduction is 0%. Skill-enabled methods can invoke composite skills, whose internal atomic calls are expanded when computing Raw.

P. Baseline Implementation Details

Table 16: Summary of baseline adaptation. “Prompt” means that the method injects memory, workflow, or skill descriptions into the prompt. “Action” means that the method exposes composite tools or skills as callable actions.

Method group	Built from training trajectories	Used at inference as	Comp./Raw reported
General tool-use agents	no	atomic tool policy	no
Memory and workflow reuse	yes	prompt guidance	no
Tool composition	yes	composite action or compiled interface	yes
Skill construction	yes	prompt skill or composite action	yes
CIPO	yes	executable composite skills + trained policy	yes

This appendix describes how we adapt baselines to the same tool-use environment. All methods use the same train/test split, backbone models, atomic tool definitions, simulator, maximum turn budget, decoding setting, and output parser. Methods that require memory, workflows, or skills build their auxiliary resources only from training trajectories. Test episodes are never used for memory construction, workflow induction, skill mining, or hyperparameter selection.

Unified evaluation interface. All baselines are evaluated as multi-step tool-use agents. At each step, the model receives the task, previous tool calls, tool observations, and available tool specifications, then outputs the next tool call in the same structured JSON format. The generated tool call is executed by the same simulator used by CIPO. For methods without executable composite actions, Comp./Raw equals 1 for trajectories with at least one tool call. Dashes in the main comparison table indicate that this value is not reported.

General tool-use baselines. ReAct is implemented as an atomic-tool agent that interleaves reasoning, tool calls, and final answers. ToolLLM is adapted as a tool-retrieval and atomic invocation baseline over the same candidate tool schema. Both methods operate over atomic tools only and do not construct memory, workflows, or composite skills.

ExpeL. We implement an ExpeL-style dual-memory baseline. Episodic memory stores training trajectories with task descriptions, tool sequences, arguments, observations, and success labels, mainly indexing successful trajectories for retrieval. Insight memory stores natural-language tool-use rules extracted by comparing successful and failed trajectories within the same task group. At evaluation time, retrieved episodic examples and ranked insight rules are inserted into the system prompt. ExpeL therefore provides both concrete examples and abstract guidance, but it does not change the action space.

MemP. MemP is implemented as a procedural-memory baseline. Each training episode is distilled into a natural-language workflow with its task query, task name, and tool sequence. Both successful and failed

samples are used; failed samples are converted through a corrective prompt into more appropriate workflows. At evaluation time, the current task retrieves the top $k = 3$ workflows by query similarity, and these workflows are injected into the prompt together with a reduced candidate tool set. MemP guides atomic tool selection through retrieved procedures.

Mem0. Mem0 is adapted as a consolidated long-term procedural memory. Each training episode is summarized into a short reusable workflow and an associated tool sequence. When a new memory is produced, similar existing memories are retrieved and an LLM decides whether to add, update, or skip the new item. At evaluation time, the system retrieves relevant procedure memories once at the beginning of each episode and keeps them fixed throughout the interaction. The retrieved memories guide tool selection but remain prompt-level knowledge.

AWM and AutoManual. For AWM, successful training episodes are grouped by task type and summarized into workflow blocks containing a workflow name, applicability condition, and ordered step templates. AutoManual similarly constructs instruction-style manuals that describe stable tool-use procedures and parameter hints. During evaluation, relevant workflows or manuals are retrieved and inserted into the prompt. Both methods reuse procedural knowledge as guidance rather than as executable actions.

Tool composition baselines. AWO is adapted by identifying repeated workflow traces from successful training trajectories and registering them as meta-tools when their internal tool sequence and interface can be determined. LLMCompiler is adapted as a planning and function-calling baseline that groups independent calls into execution stages and executes dependent stages sequentially in our simulator. LLM-Tool Compiler is adapted by exposing fused interfaces for similar or frequently co-occurring tools. These baselines introduce higher-level interfaces or compiled execution structures, but they do not train atomic-versus-skill choices with matched counterfactual branches.

Skill construction baselines. SkillRL is adapted as a skill-bank baseline that distills training trajectories into reusable skill descriptions and, when possible, exposes concrete tool subsequences as callable composite actions. SKILL0 is adapted as a skill-internalization baseline, where skill descriptions are provided during training and gradually reduced so that the policy internalizes their use. Trace2Skill extracts trajectory-local lessons from training traces and consolidates them into a skill directory; retrieved skills are used as prompt guidance, and executable subsequences are exposed as composite actions when their schema can be inferred. These methods reuse skills, but they do not provide CIPO’s first-eligible branch comparison or paired reward difference for granularity selection.

Internal GRPO baselines. SFT denotes the skill-enabled supervised baseline. Standard GRPO starts from a separate atomic-only SFT model and uses only atomic tools during reinforcement learning and evaluation. GRPO + Skills starts from the skill-enabled SFT model and uses the augmented action space without counterfactual branching. Appendix [O](#) details these configurations. The w/o Budgeted Mining variant replaces constrained mining with a less controlled skill library, while keeping the rest of the pipeline unchanged. CIPO-random uses the same group size $G = 4$ as CIPO-first, but samples the branch point from all eligible granularity decisions instead of using the first eligible decision.

Fairness considerations. All baselines are compared under the same model backbones, evaluation tasks, tool simulator, decoding budget, and metric definitions. Memory and workflow methods may use training trajectories, but only as prompt-level guidance. Composite-action methods may register additional callable actions, but their internal executions are expanded when computing Raw. This setup ensures that differences in performance reflect the method-specific use of memory, workflow reuse, tool composition, skill construction, or granularity training rather than differences in the evaluation environment.

Q. Broader Impact

CIPO studies how LLM agents choose between atomic tools and composite skills during long-horizon tool use. The main positive impact is improved efficiency and reliability for tool-use agents. By compressing recurring tool-use routines into executable skills, CIPO can reduce repeated low-level decisions and help agents complete complex workflows with fewer policy-level actions. This can make tool-use systems more practical in settings such as research assistance, software workflows, document processing, and API-based data analysis.

The same capability also requires care. More effective tool-use agents may execute longer or more complex action sequences, and errors in tool selection can propagate through downstream tool calls. Composite skills may also hide intermediate steps if they are implemented as opaque pipelines. CIPO mitigates this issue by making skills executable, trace-aware, and interruptible: each internal step can be recorded, inspected, and interrupted when execution fails. The counterfactual reward is also bounded, which reduces the risk that the policy over-optimizes for skill invocation at the cost of task completion.

Our experiments are conducted in benchmarked tool environments rather than deployed high-stakes settings. We do not evaluate or recommend direct deployment in domains where erroneous tool execution could cause material harm, such as medical, legal, financial, security-critical, or safety-critical decision making. In such settings, CIPO-style agents should be combined with human approval, sandboxed execution, tool permission control, logging, and domain-specific safety checks. Future work can further study online monitoring, dynamic tool updates, and safeguards for preventing unsafe or unintended composite skill execution.